

目录

1	目的	- 3 -
2	适用范围	- 4 -
3	工具介绍	- 5 -
3.1	观测标定工具	- 5 -
3.2	总线检测工具	- 5 -
4	前提条件	- 6 -
4.1	软件配备	- 6 -
4.2	硬件配备	- 6 -
4.3	驱动配置	- 6 -
5	A2L 文件	- 8 -
5.1	编辑软件	- 8 -
5.2	导入 ELF	- 8 -
5.3	添加观测或标定变量	- 11 -
5.4	更新 A2L	- 11 -
6	工程配置	- 13 -
6.1	创建工程	- 13 -
6.2	查看通道配置	- 16 -
6.3	加载 A2L 文件	- 16 -
6.4	传输协议配置	- 20 -
6.5	内存分布配置	- 25 -
6.6	数据冻结（部分项目支持）	- 29 -
7	INCA 的 A2L 制作说明	- 32 -
8	文件修订履历表	- 33 -

1 目的

本文档目的在于帮助测试，标定人员快速掌握该套 XCP 标定观测，工具的使用方法。

2 适用范围

本文档主要说明使用 CANape16Sp6 和 VN1630 或 VN1640 进行观测，标定操作，意在通过图示可以简单，直观的指导标定人员进行观测和标定的工作。

3 工具介绍

3.1 观测标定工具

目前我司使用的观测标定工具主要是 CANape16SP6。

3.2 总线检测工具

目前我司主要使用的总线检测工具为 VN1630, VN1640。

4 前提条件

4.1 软件配备

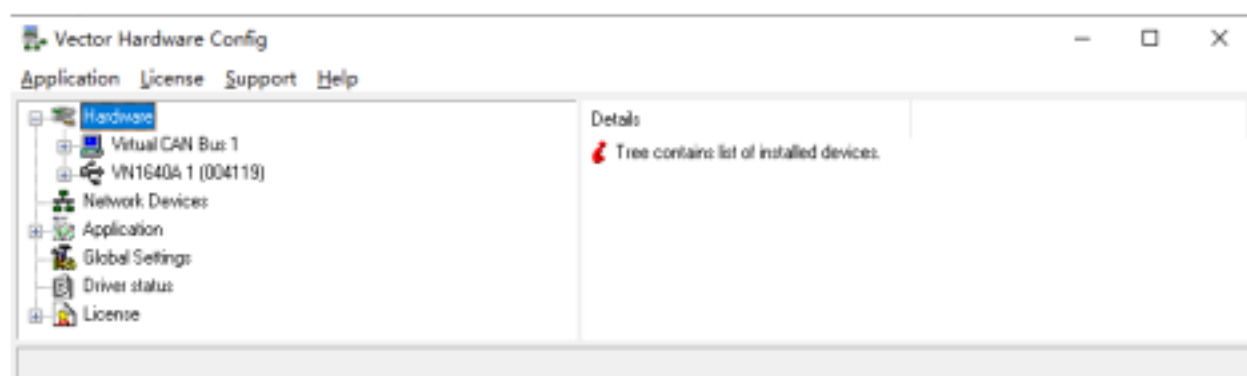
标定人员需要准备对应项目的 ELF 和 HEX 文件,使用的计算机需要安装 CANape16SP6, VN1630 或 VN1640 驱动。

4.2 硬件配备

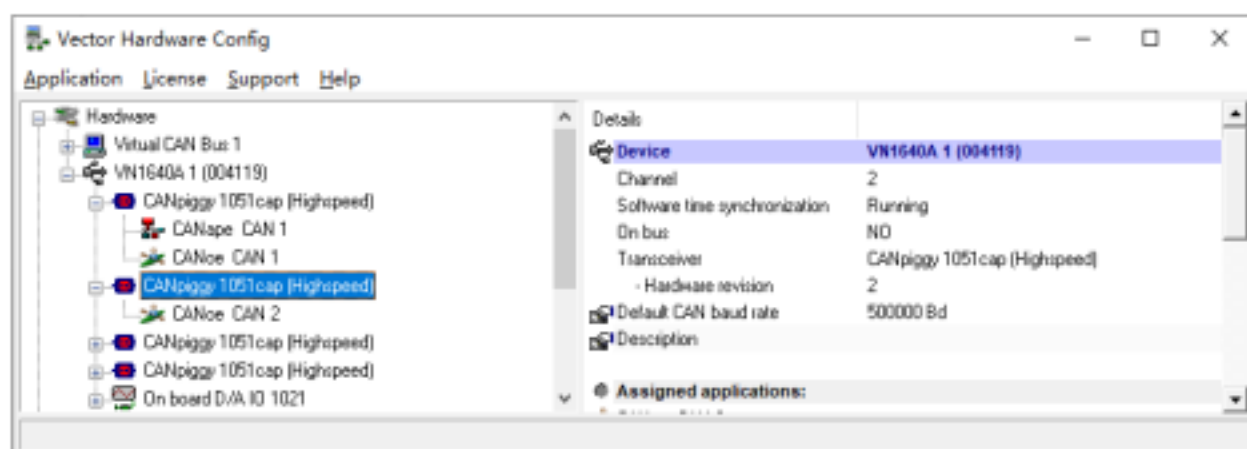
标定人员需要确认产品正确的标定 CAN 口, 正确的连接到 VN1630 或者 VN1640 的通道上。**注意: VN1630 带有扩展 CAN 口必须使用 1 转 2 的线束才可以。**

4.3 驱动配置

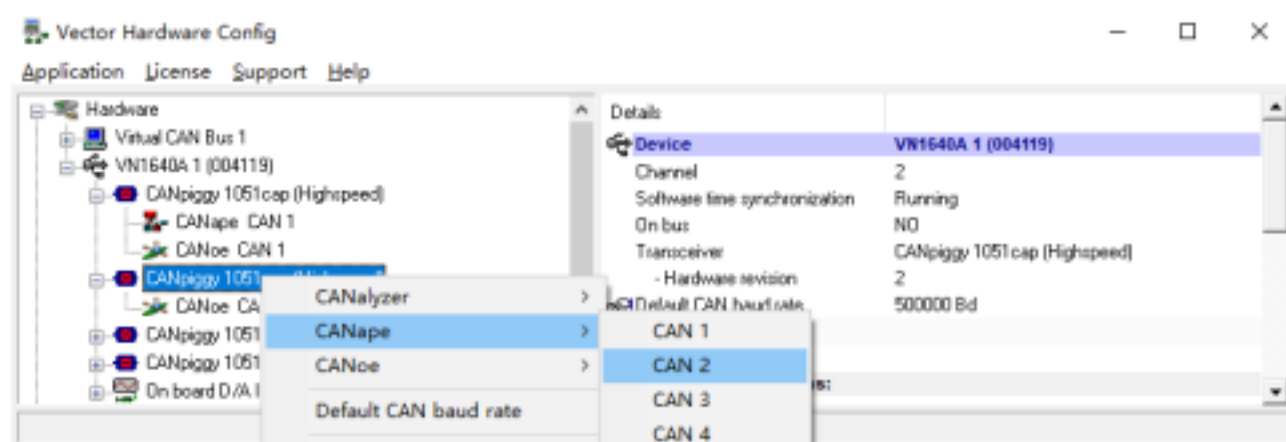
在安装完驱动的计算机中, 打开控制面板可以找到 Vector hardware configuration 应用, 双击打开如下图所示



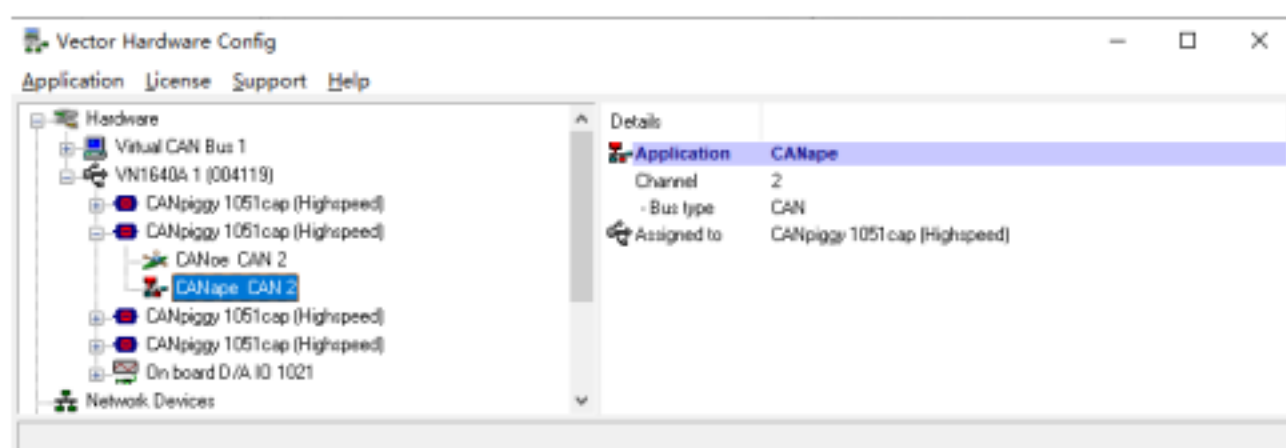
选择真实的硬件 VN1640 或 VN1630, 点击 “+” 查看每个通道的配置, 图示如下:



检查每个通道是否按顺序分配到 CANape 软件通道中, 如上图所示的第二通道没有分配到 CANape 的 CAN2 通道, 选中该通道右键点击进行分配, 如下图所示:



分配后，驱动配置部分已经完成。如下图所示：



5 A2L 文件

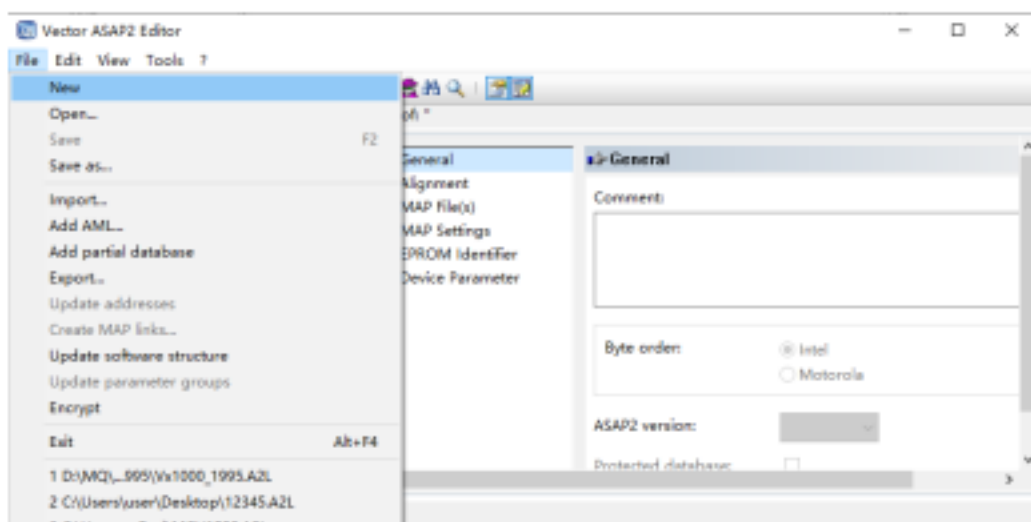
A2L 文件是完成标定工作的重要资源，需要进行生成和更新。以下步骤将详细描述 A2L 文件的制作方法。

5.1 编辑软件

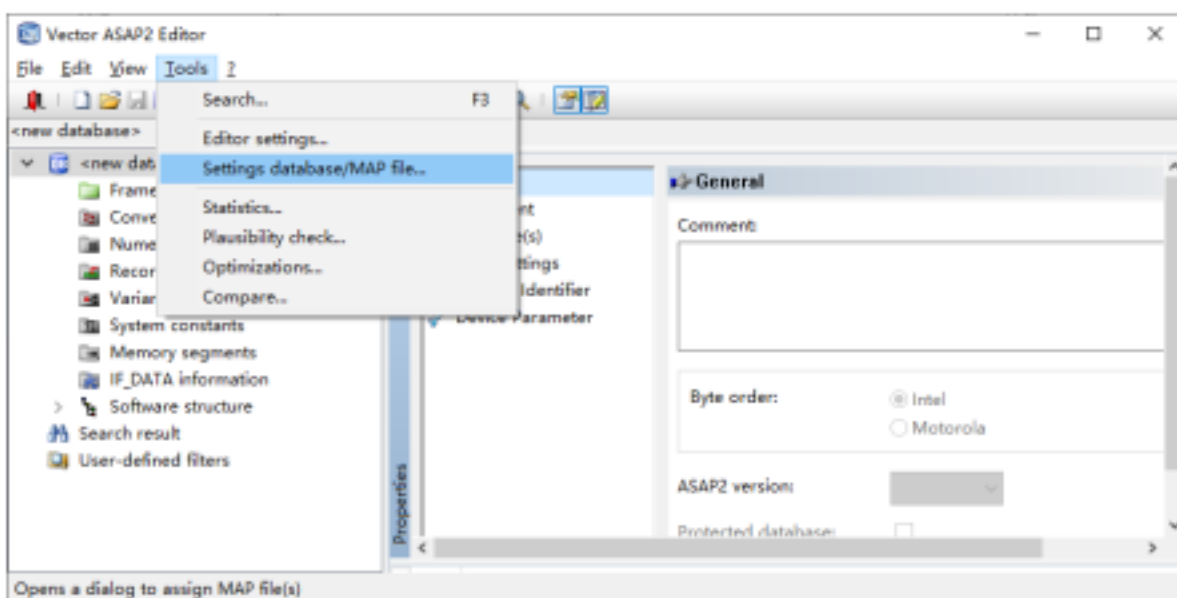
我司使用 CANape16SP6 工具目录中的 ASAP2 Editor 进行 A2L 文件的编辑。如若不修改默认安装目录该软件是 C:\Program Files (x86)\Vector\ASAP2 Editor\Exec\edita2l.exe。

5.2 导入 ELF

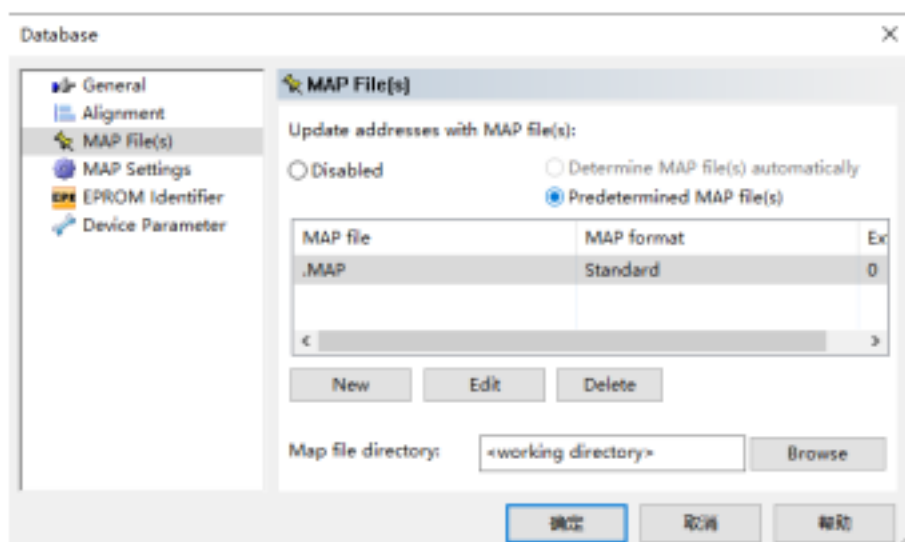
第一步：打开 ASAP2 Editor，创建一个新的 A2L 模板，具体操作为点击工具栏中的 file->new 即可，如下图所示：



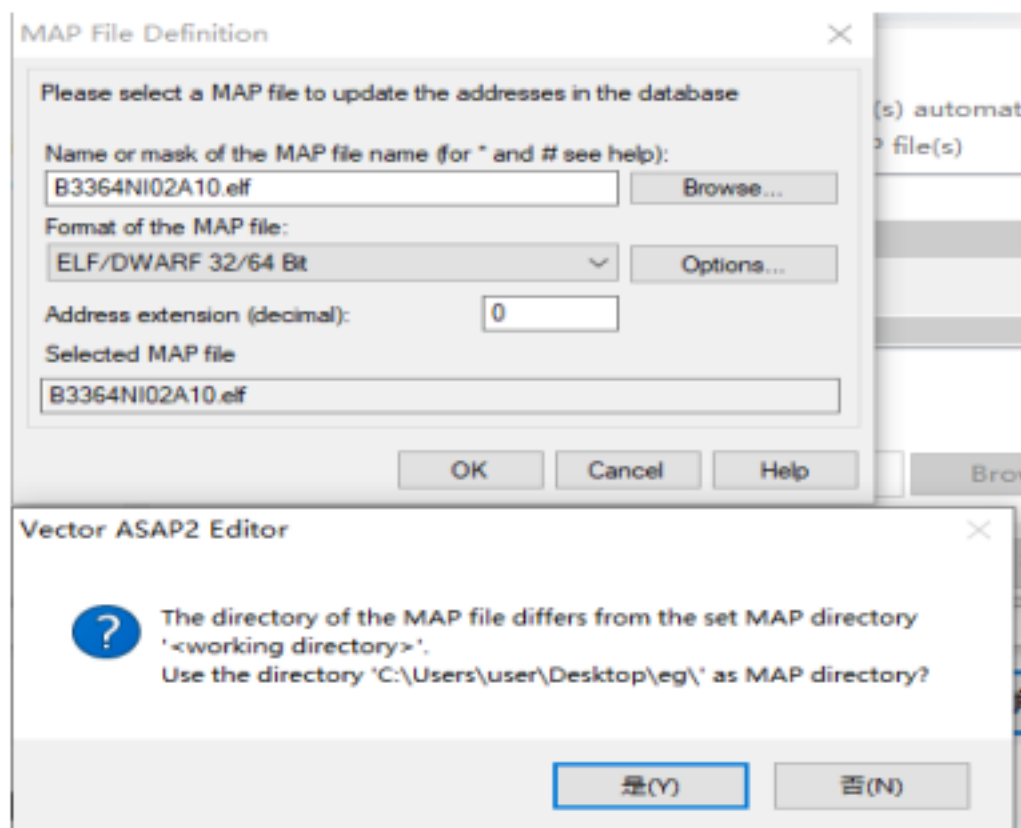
第二步：点击 Tools->Settings Database/MAP File



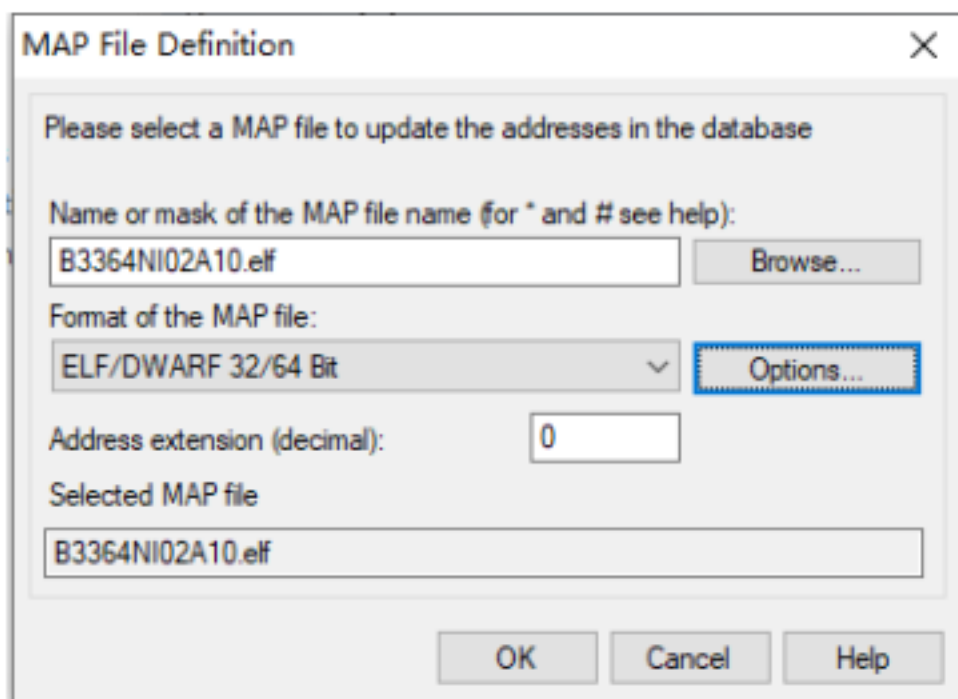
第三步：在弹出 Database 界面中选择 Predetermined MAP file(s)



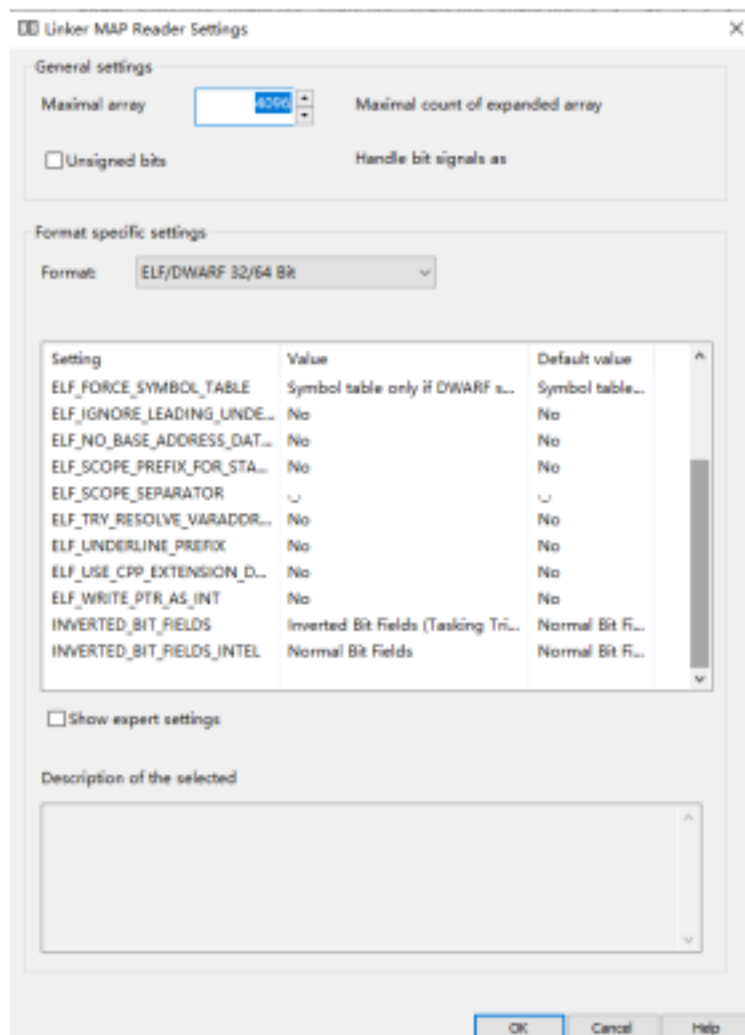
第四步，在 MAP File Definition 窗口中，点击 Browse 选择对应项目的 ELF 文件，若弹出 VectorASAP2 Editor 窗口选择“是”即可，将 Format of the MAP file 选择为 ELF/DWARF 32/64 Bit。



第五步，选择 Options 选项，进行配置数组最大长度和位域解析方式。

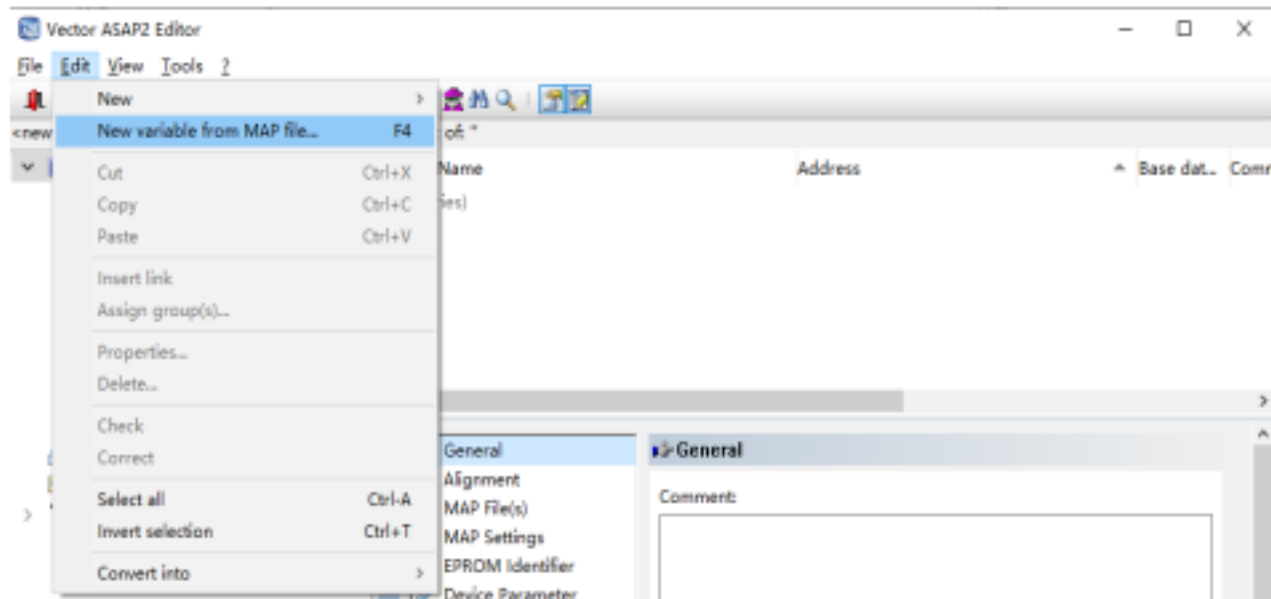


Maximal array 长度根据观测或标定需要的最大长度进行设定，INVERTED_BIT_FIELDS 必须选择 Inverted Bit Fields (Tasking Tricore Vx) 来保证位域解析的正确性。设置完成后只需点击 OK。完成关于 MAP 文件导入的设定后，退回到最开始打开的主界面。

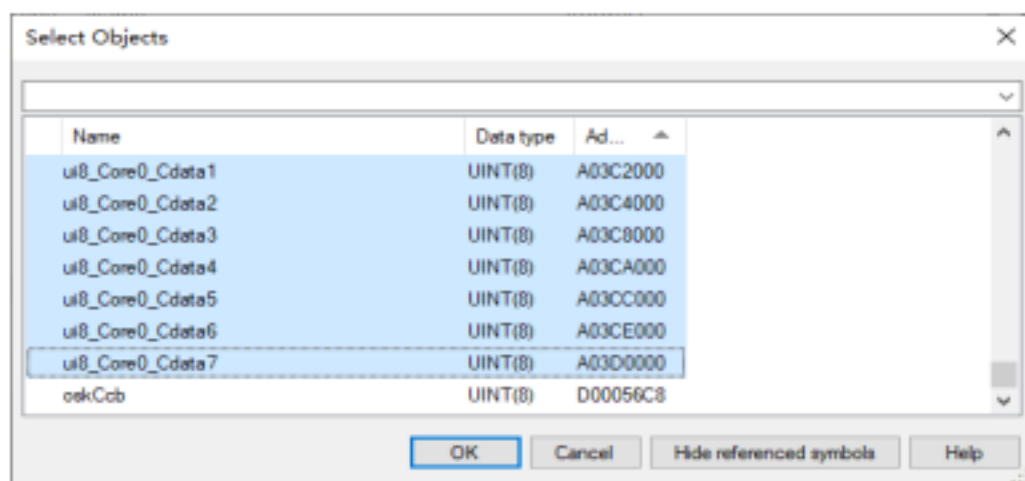


5.3 添加观测或标定变量

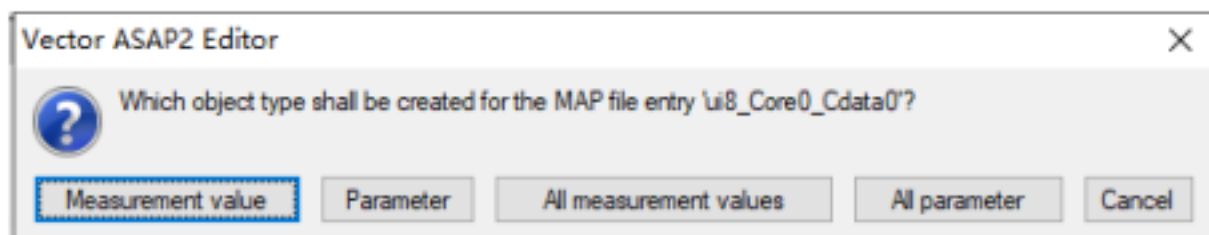
点击 Edit, 选择 New Variable from MAP file...



在弹出的 Select objects 界面中选择需要观测或标定的变量，点击 OK。



在弹出的窗口中，选择将其设定为观测量还是标定量。



完成观测量和标定量的添加以后，点击保存，即完成了 A2L 文件的制作。

5.4 更新 A2L

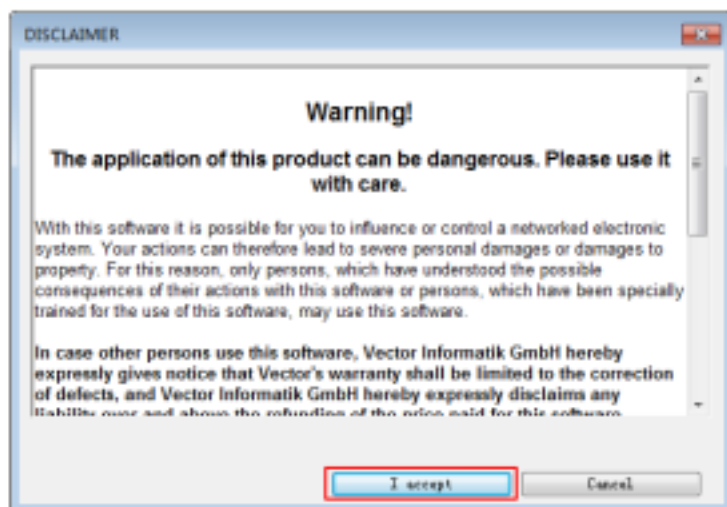
大部分标定人员都会拿到一个基础的 A2L 文件，但不避免一些情况下 A2L 文件中的数

据地址与 MAP 文件中不一致，此时我们仅需要双击打开 A2L 文件，在 ASAP2 Editor 中，进行重新导入 MAP 文件的操作即可，具体操作参考 5.2 章节。

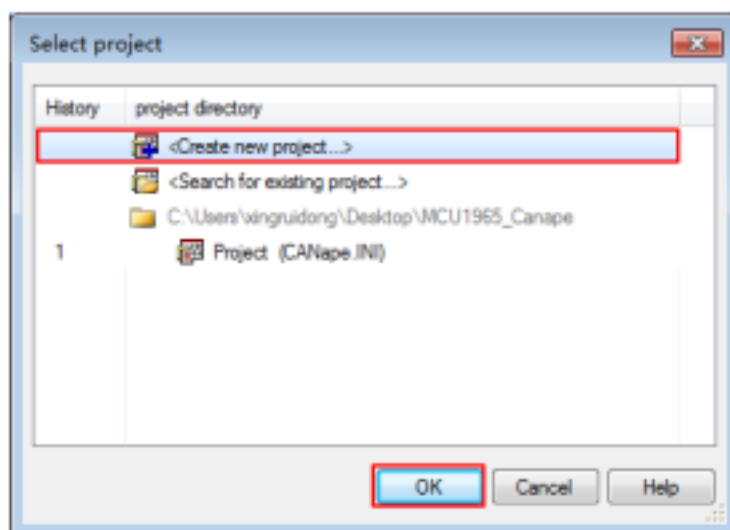
6 工程配置

6.1 创建工程

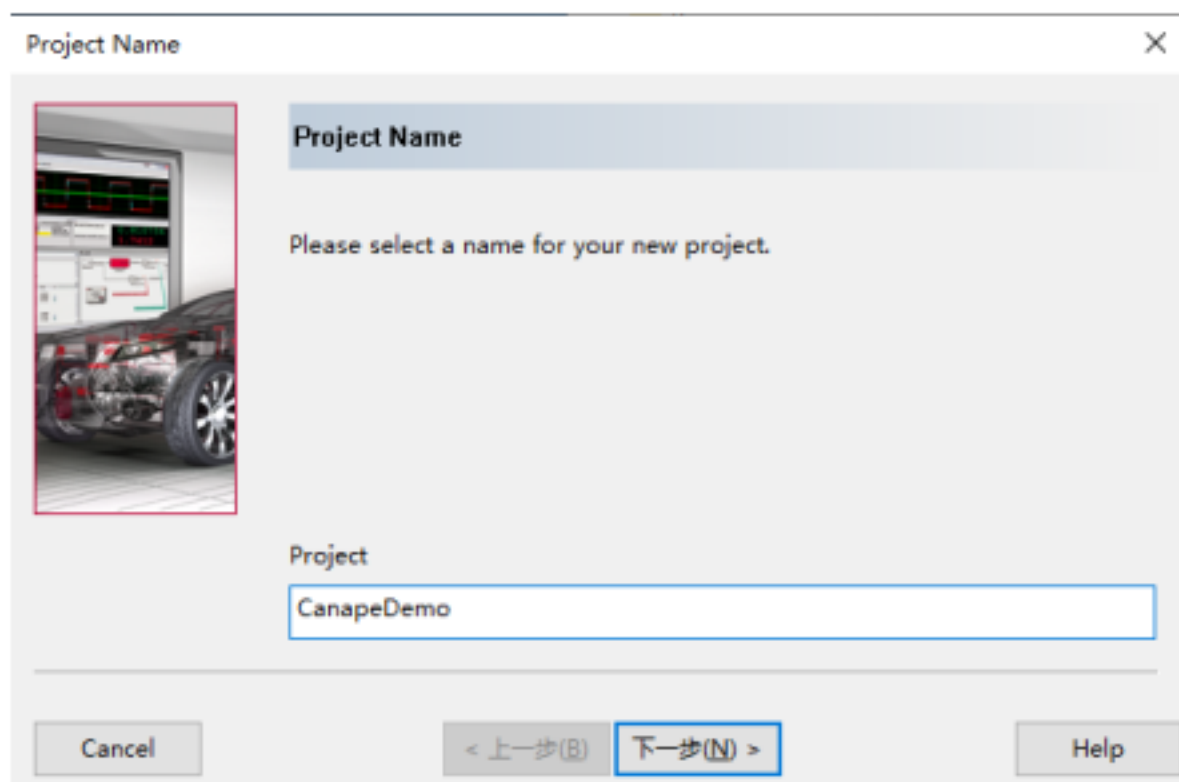
1, 打开 CANape16.0, 警告界面选择 “I accept”, 然后进入下一步。



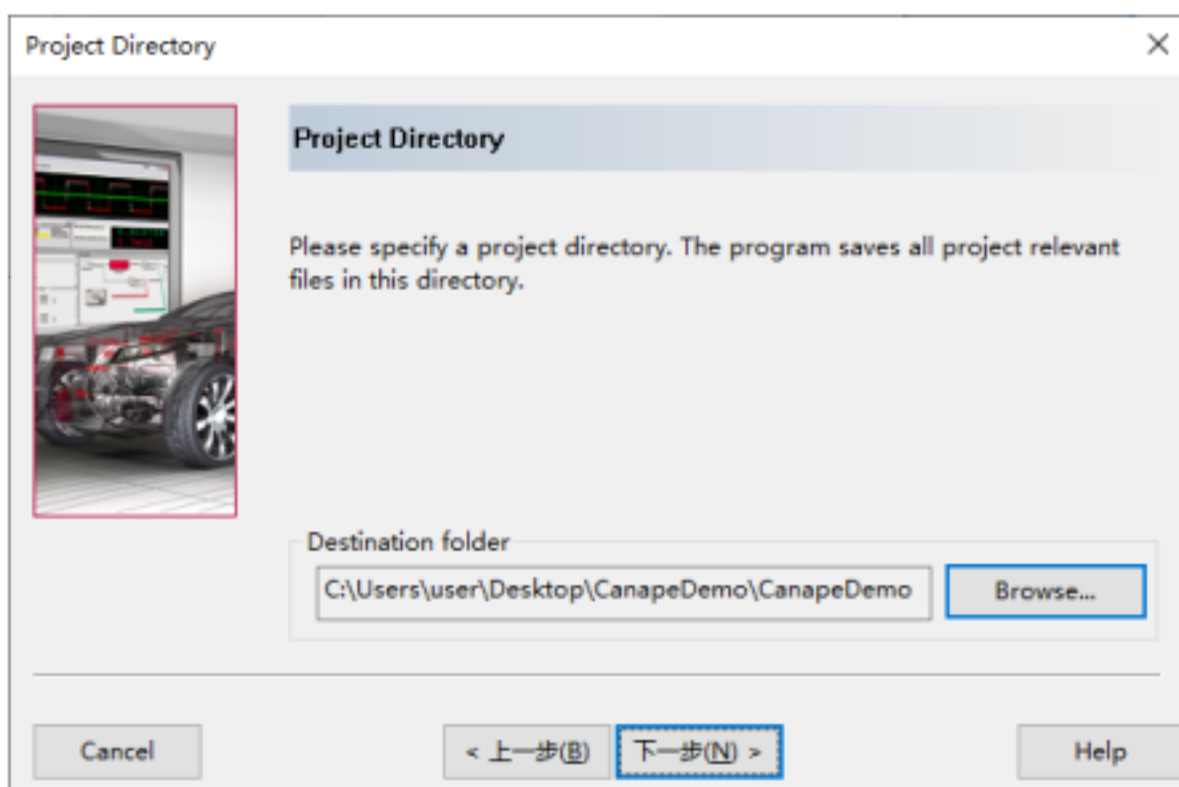
2, 选择 Create new project 来建立新工程, 点击 OK 后进入下一步。



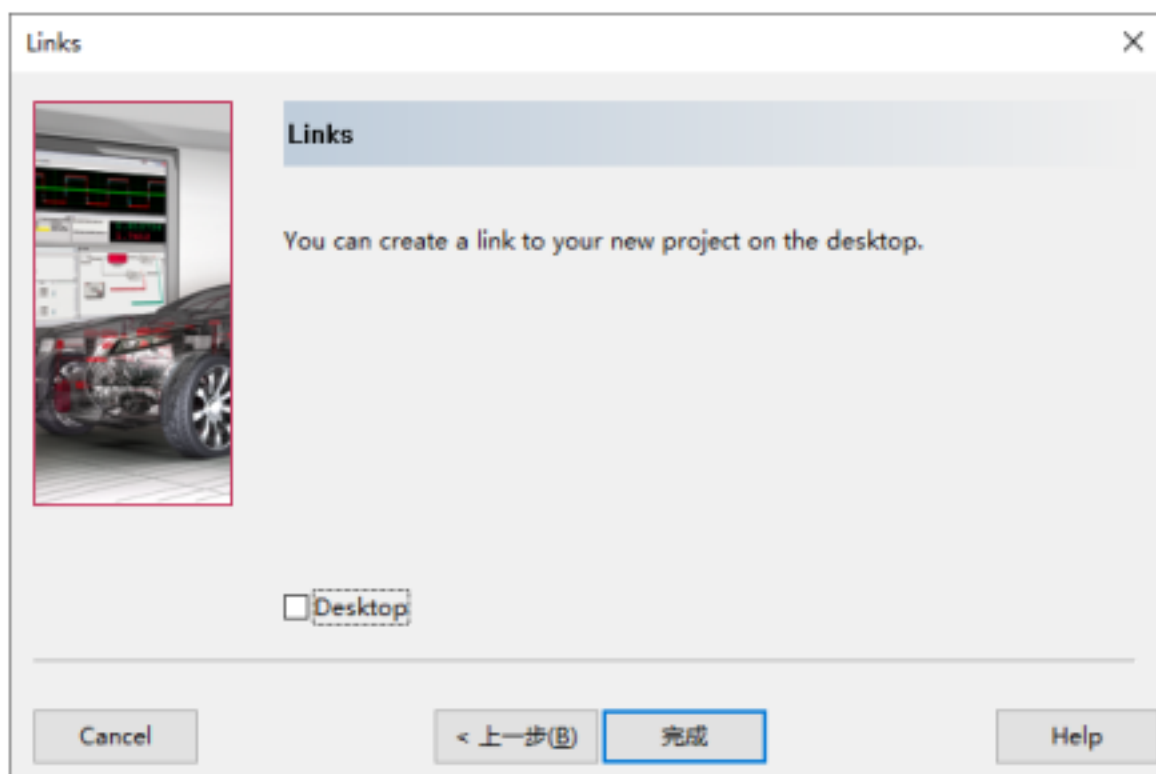
3, 给工程命名, 暂命为 “CanapeDemo”, 然后单击 “下一步”。



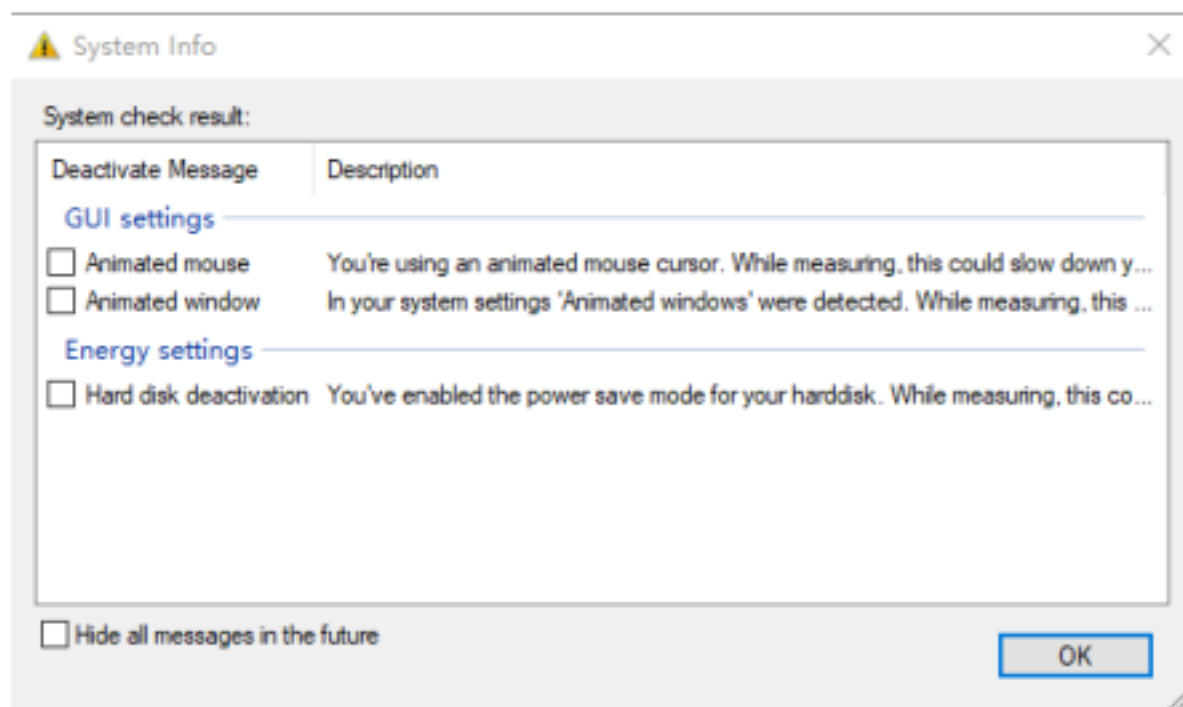
4, 选择文件夹路径, 注意此项不需要建立工程文件夹, 在选择路径之后, 软件自动创建文件夹。单击下一步。



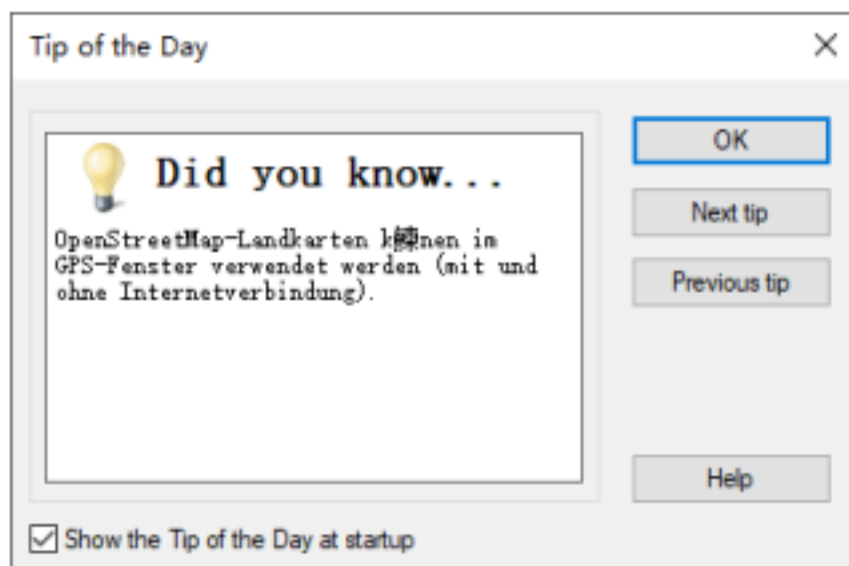
5, 点击完成, 工程新建完成。



6, 在弹出的系统信息栏中, 点击 OK。

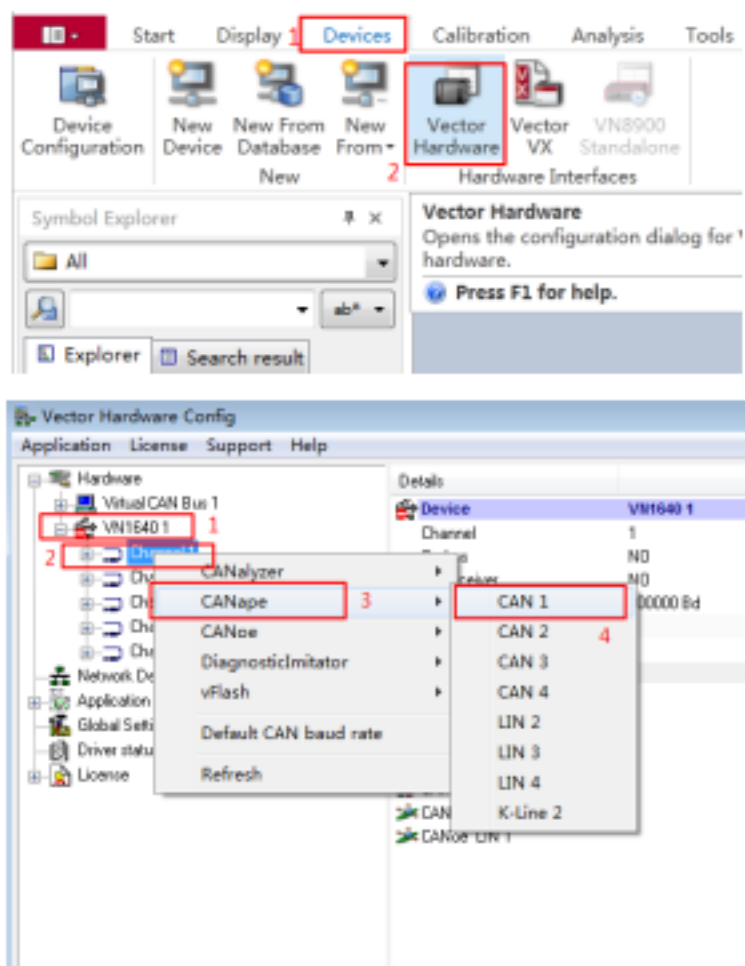


7, 点击 OK。



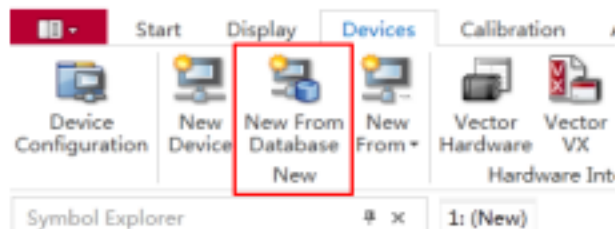
6.2 查看通道配置

1, 点击选择 Device->Vector hardware, 弹出 Vector Hardware Config 界面, 如图将所使用的硬件 (这里用 VN1640 为例) 的 channel 配置为 CANape CAN 1 通道 (此处需要根据实际硬件使用的情况进行配置)。

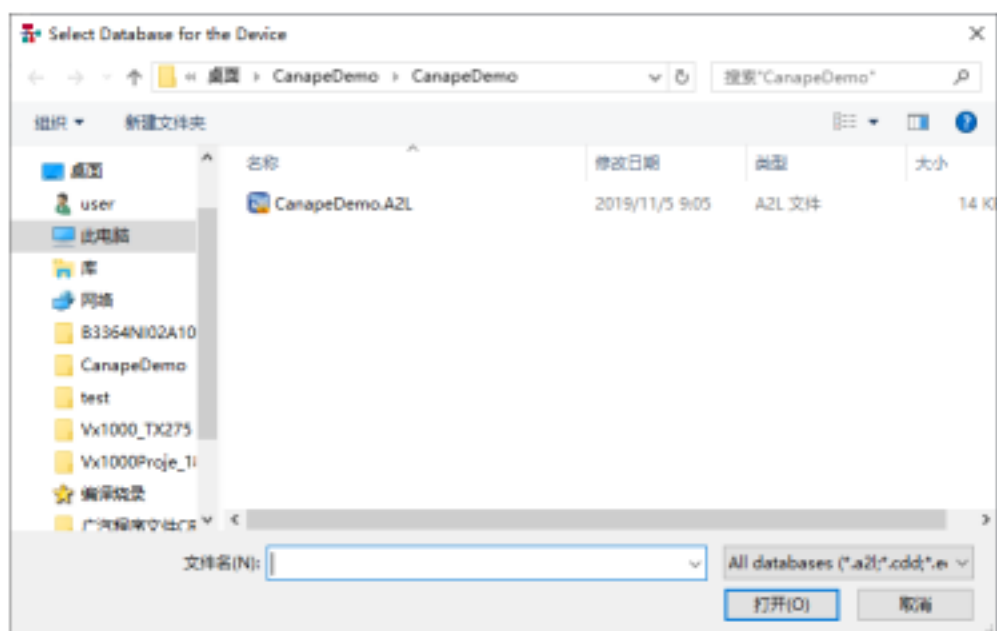


6.3 加载 A2L 文件

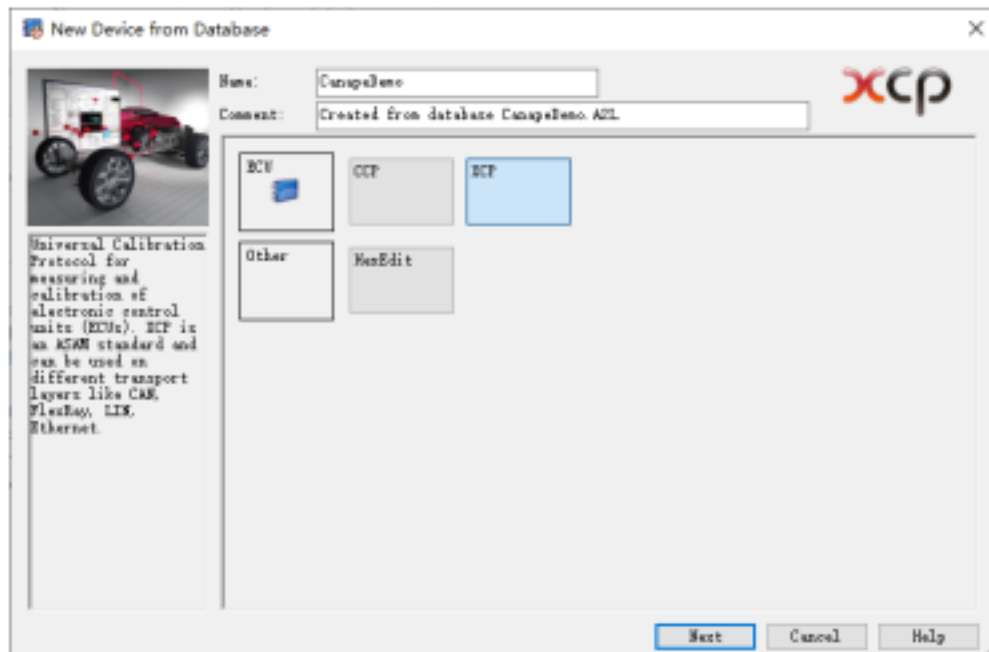
1, 加载 A2L 数据库, 点击 Device->New from Database。



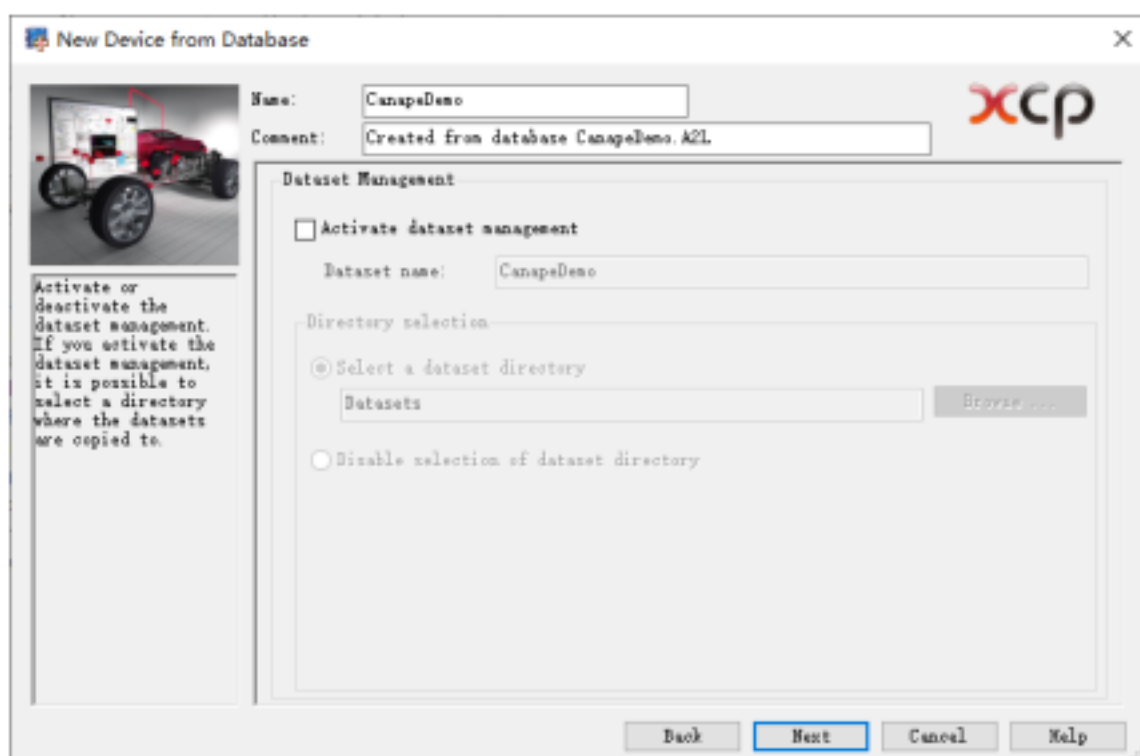
2, 选择刚才制作好的 A2L 文件, 单击打开。



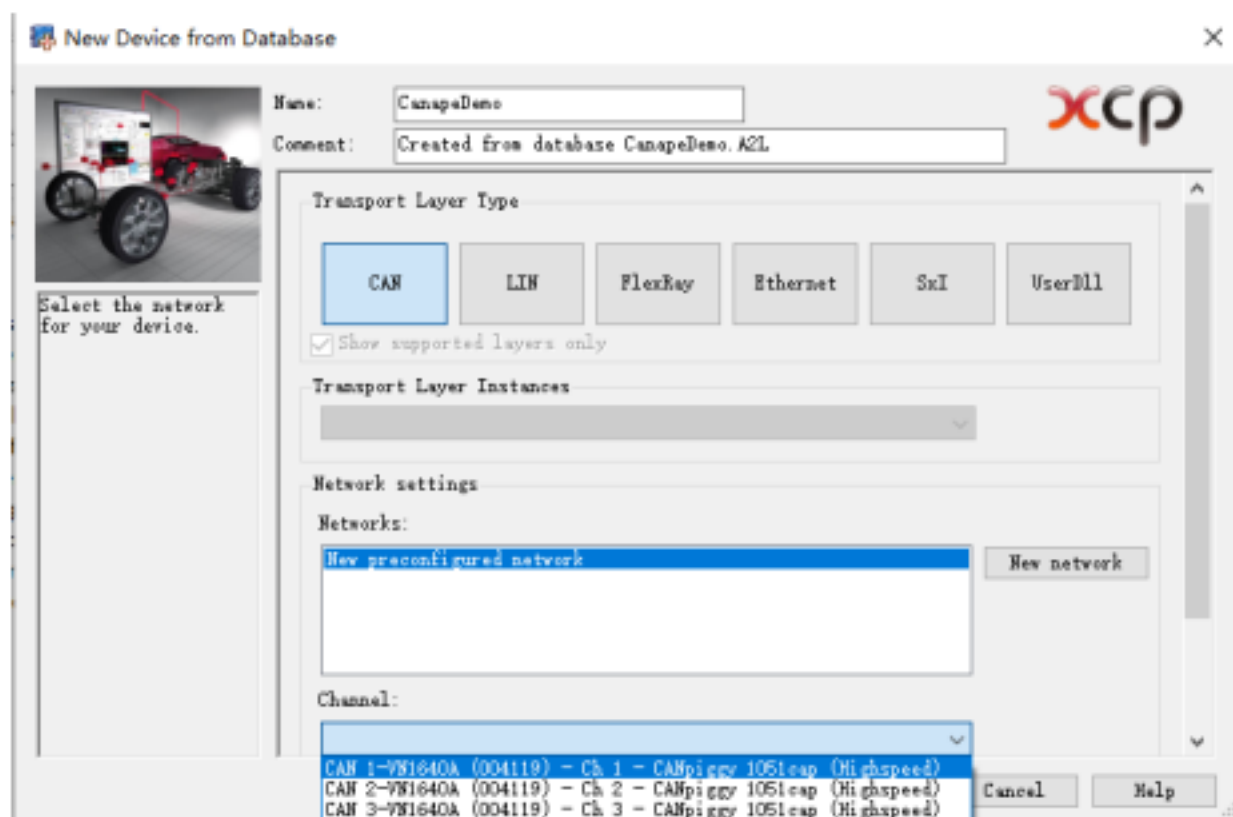
3, 在弹出的窗口中选择 XCP, 然后点击“Next”。



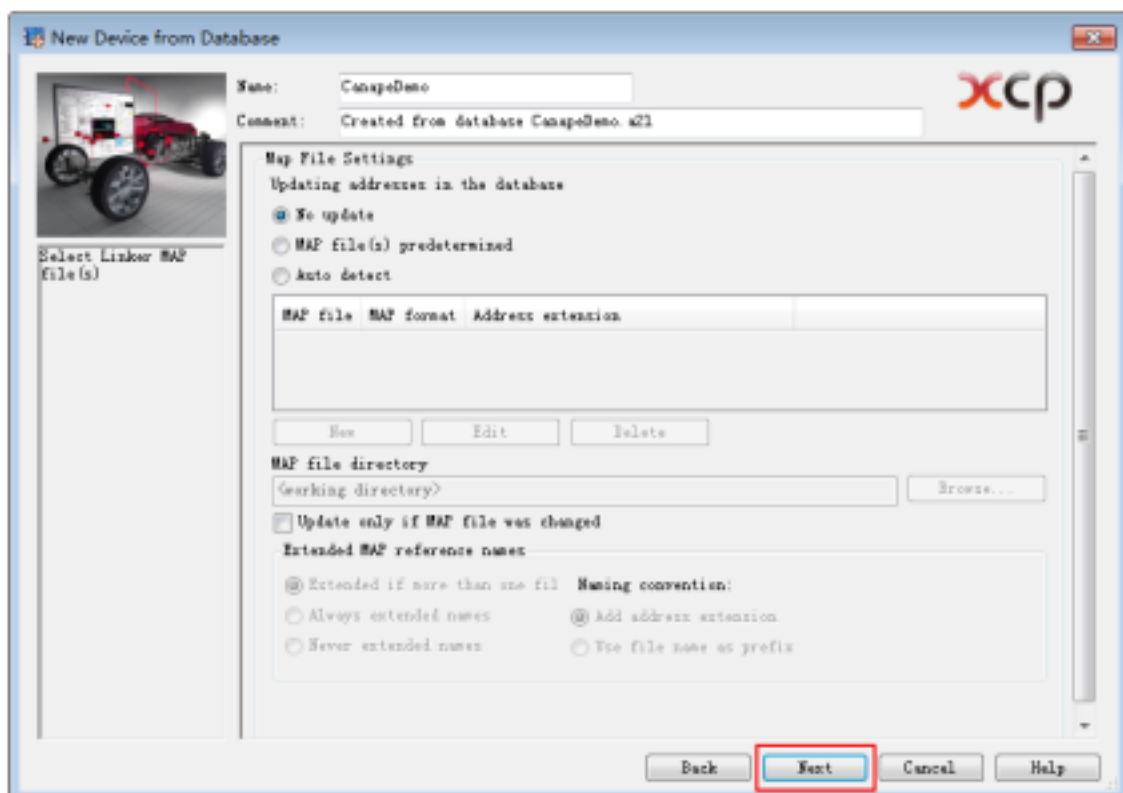
4, 单击 Next。



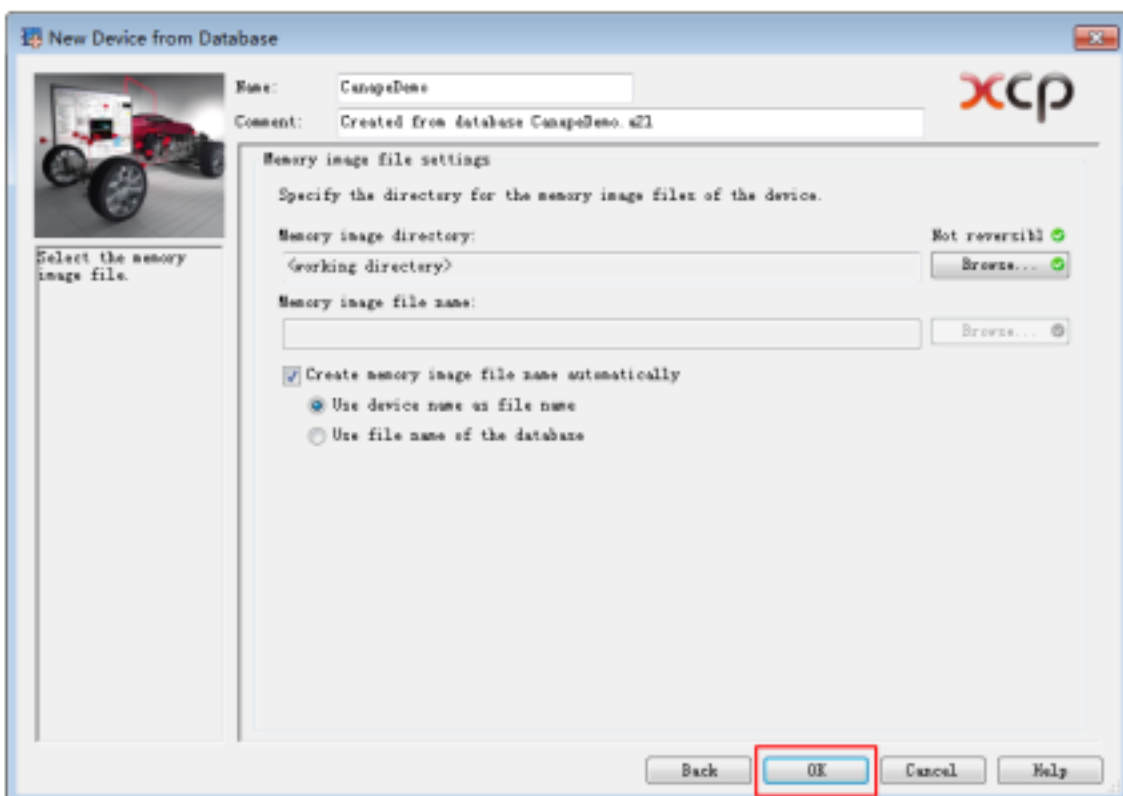
5, 选择连接的 CAN 口, 此处选择分配的通道 ch1 (此处需要根据实际分配通道来选择), 选择好后点击 “Next”。



6, 单击 Next

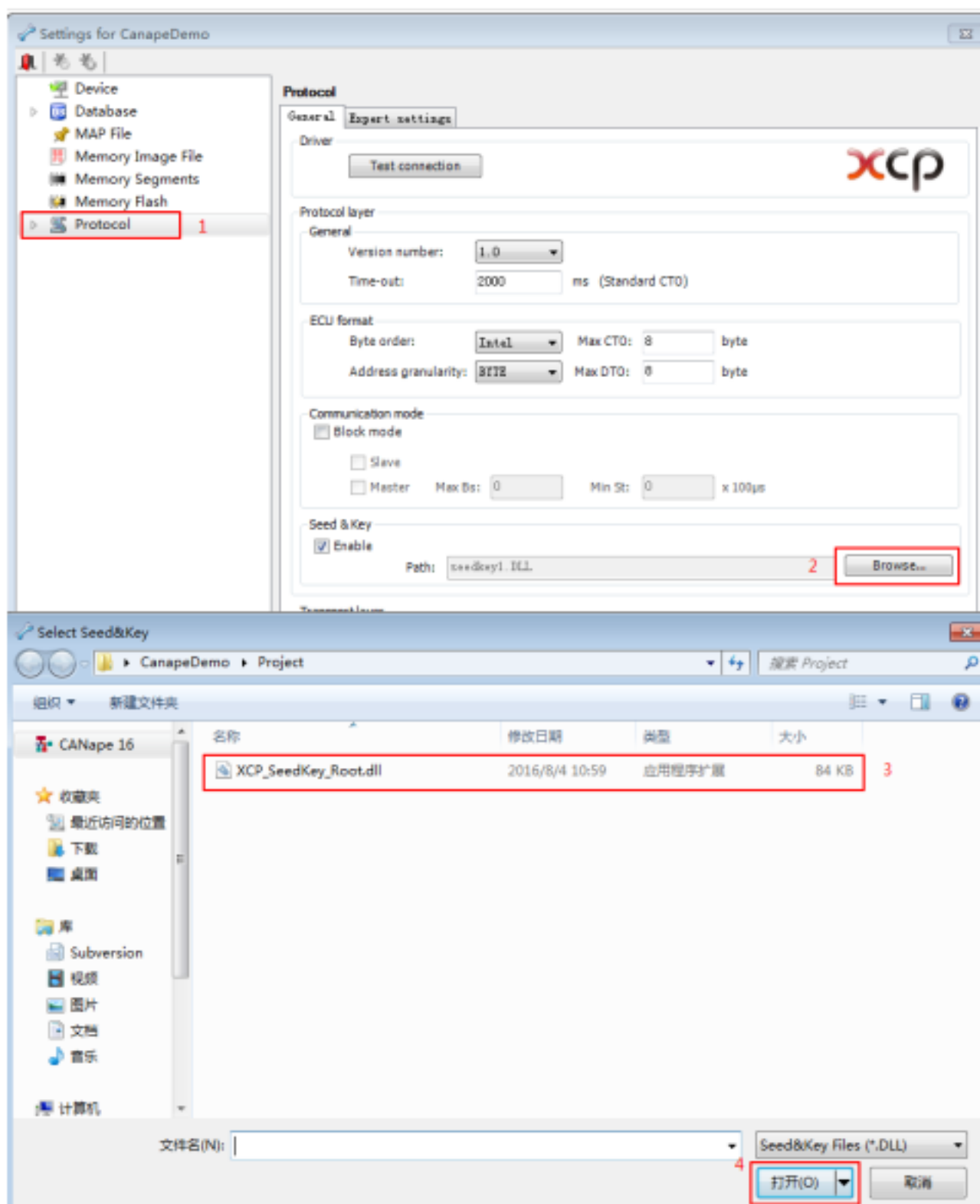


8, 单击 OK。



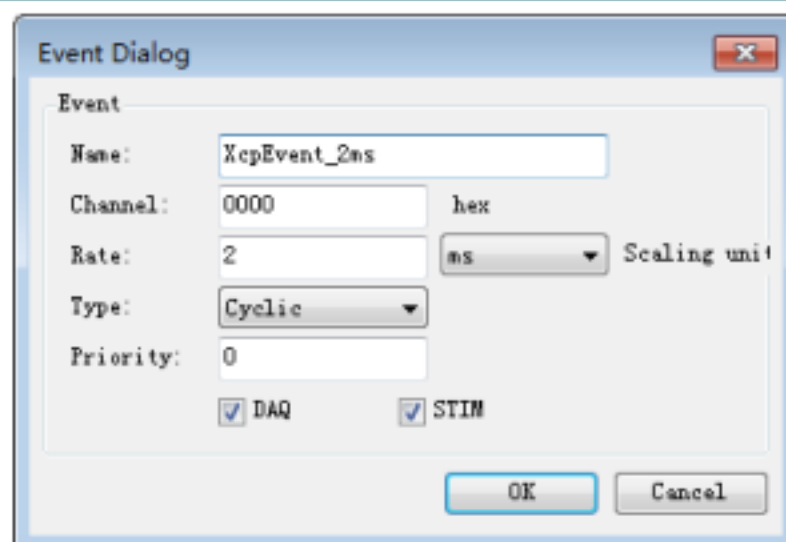
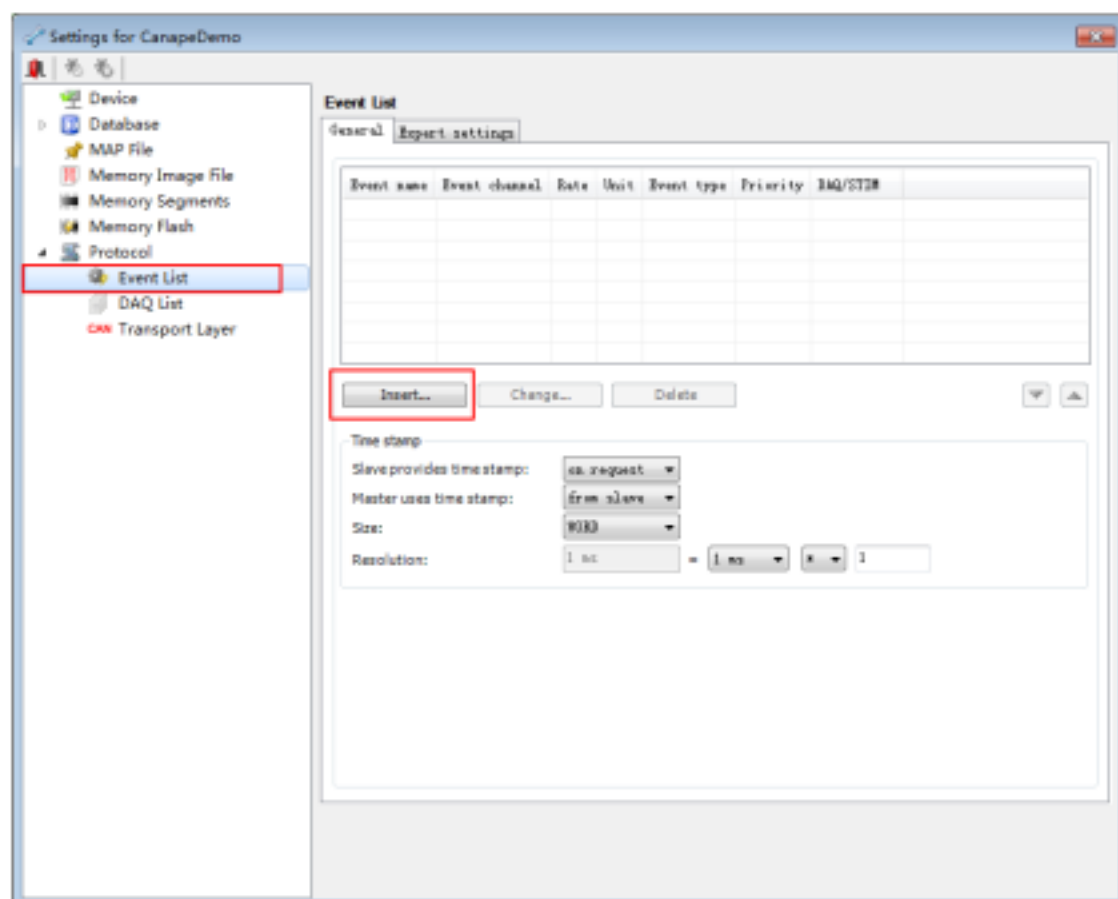
9, 在弹出的窗口中, 选择 Protocol 再选 Brwose, 选择 DLL 文件, 然后点击“打开”。

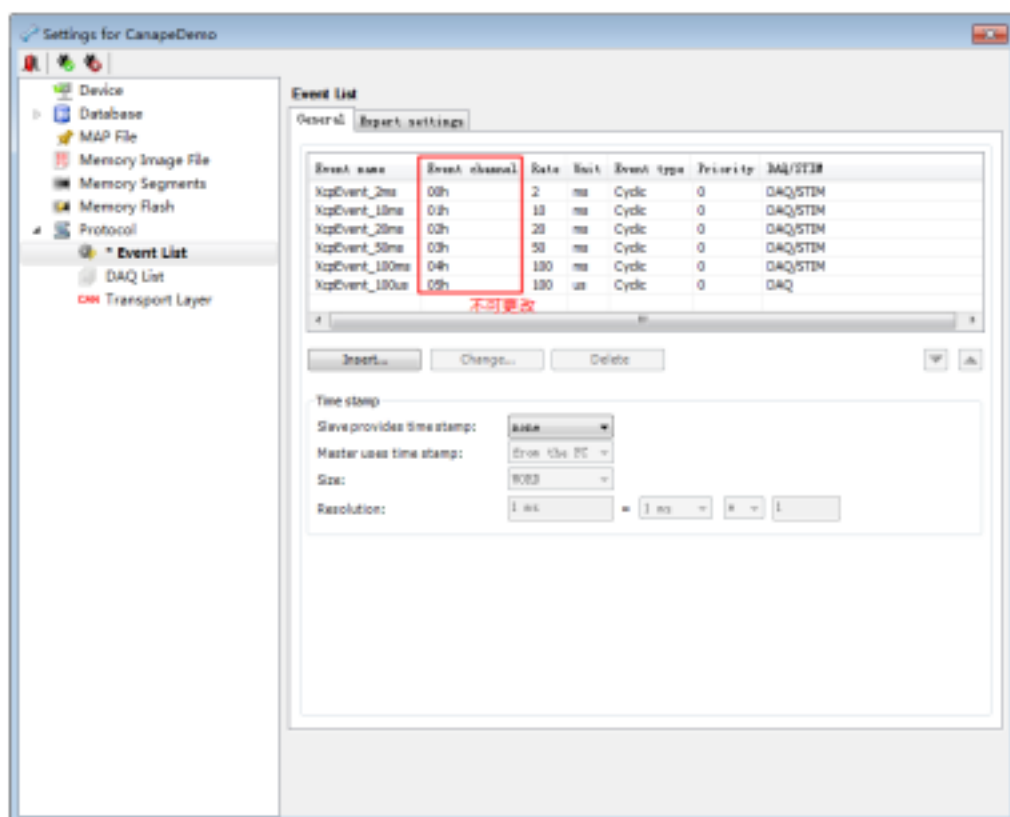
注意: 如若没有 Seed&key 校验, 不需要执行此步骤。



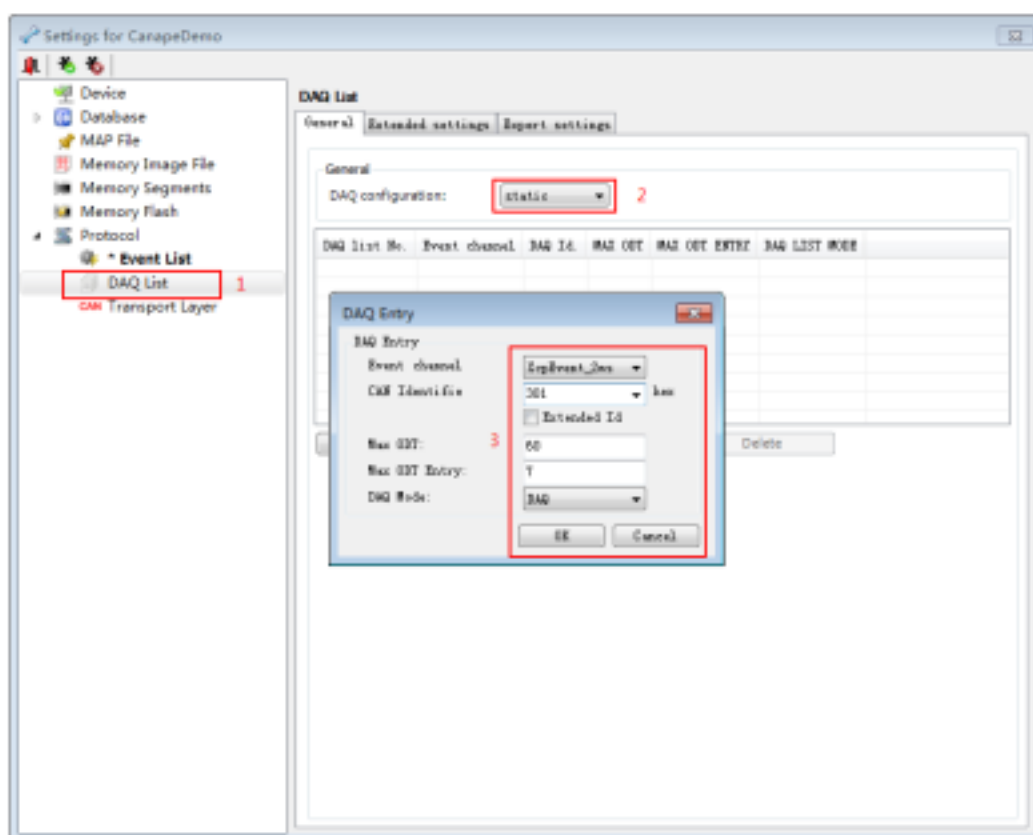
6.4 传输协议配置

1, 展开 Protocol, 点击 Event List, 单击 Insert, 添加事件通道, 根据项目具体观测通道分配来进行设置。**注意: 此配置仅对静态 DAQ 模式需要配置, 动态 DAQ 模式不需要配置此处。**

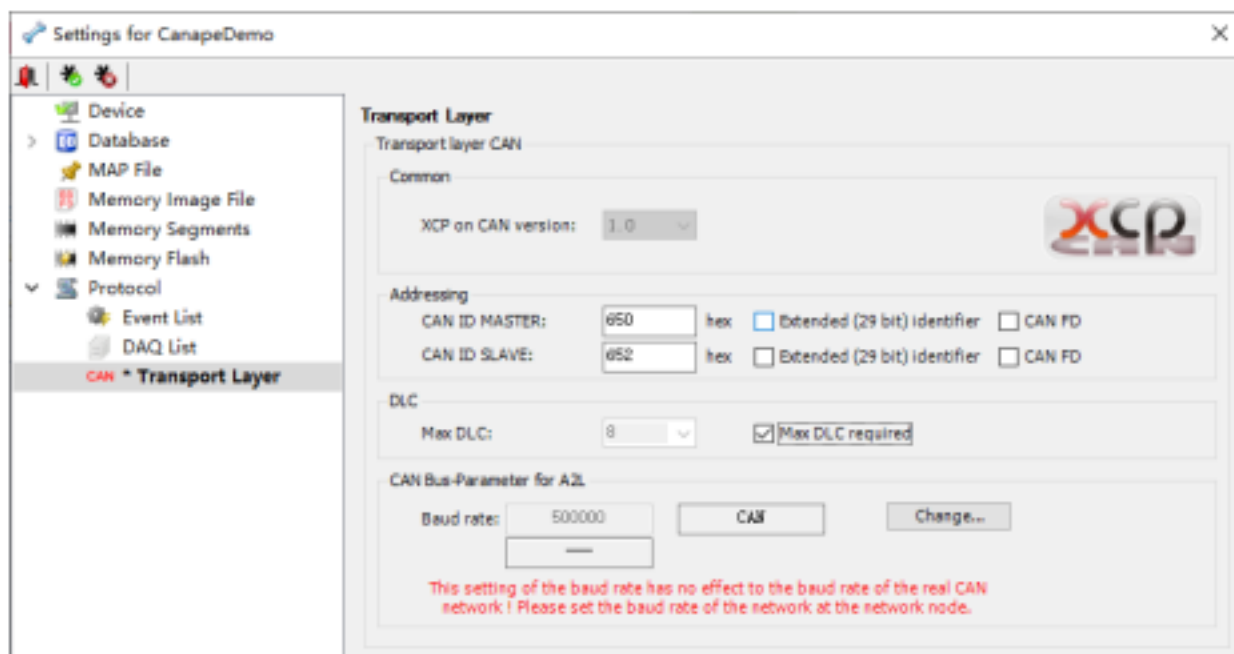




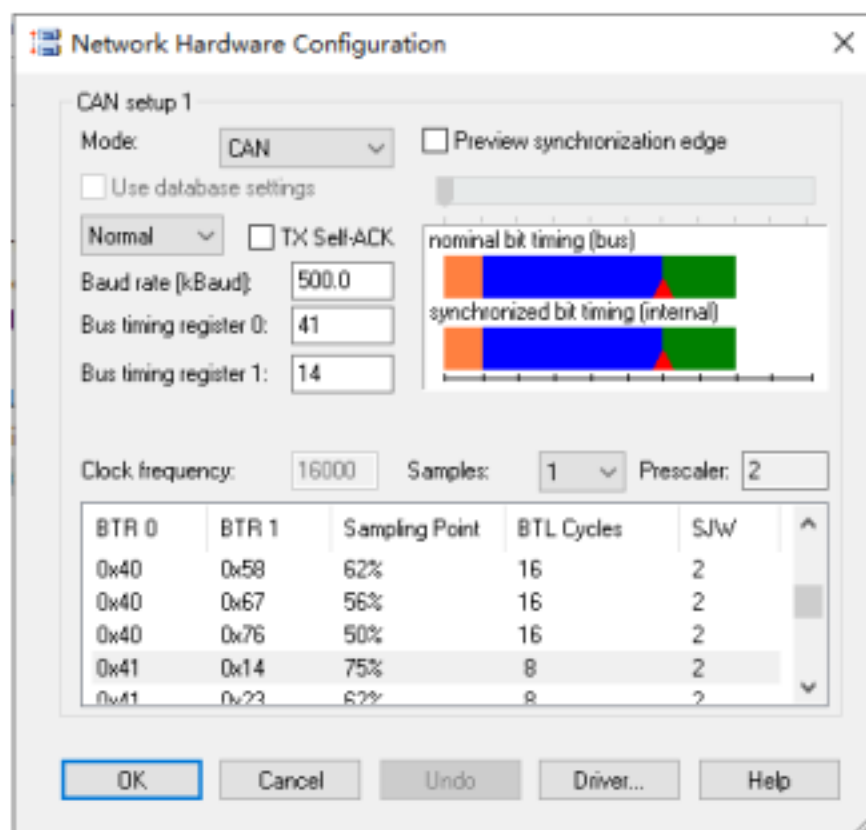
2, 点击 DAQ List, 如图设置。DAQ ID 不可更改。注意: 动态 DAQ 分配模式不需配置此处。手写代码 XCP 工程都是用静态 DAQ, 工具链配置的都是用动态 DAQ。



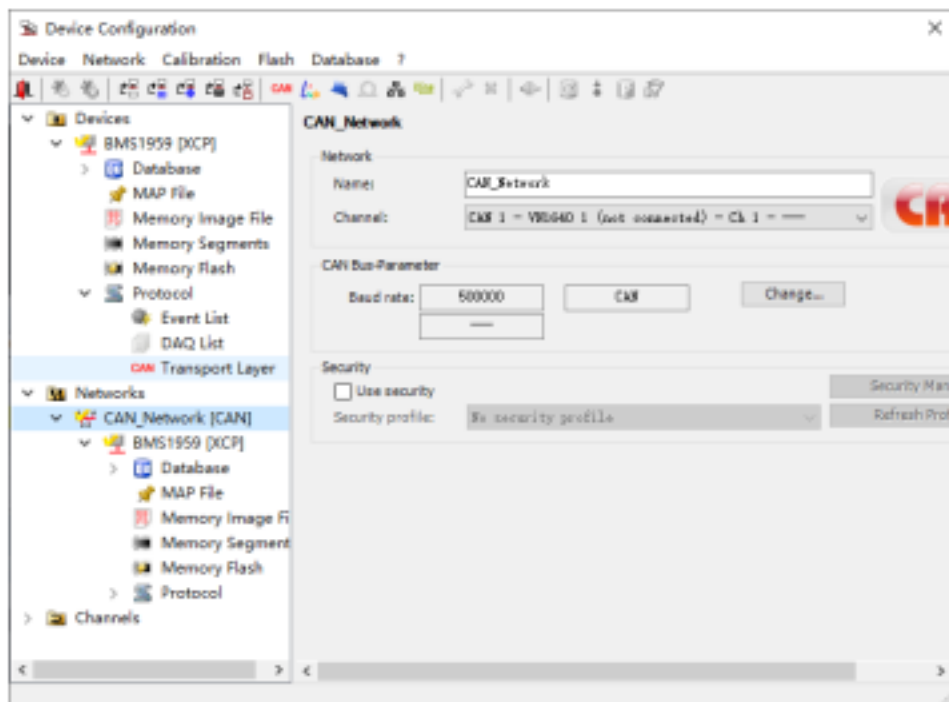
3, 点击 Transport Layer, 如下设置 (需根据实际工程配置 ID 和波特率)。MAX DLC 必须勾选。



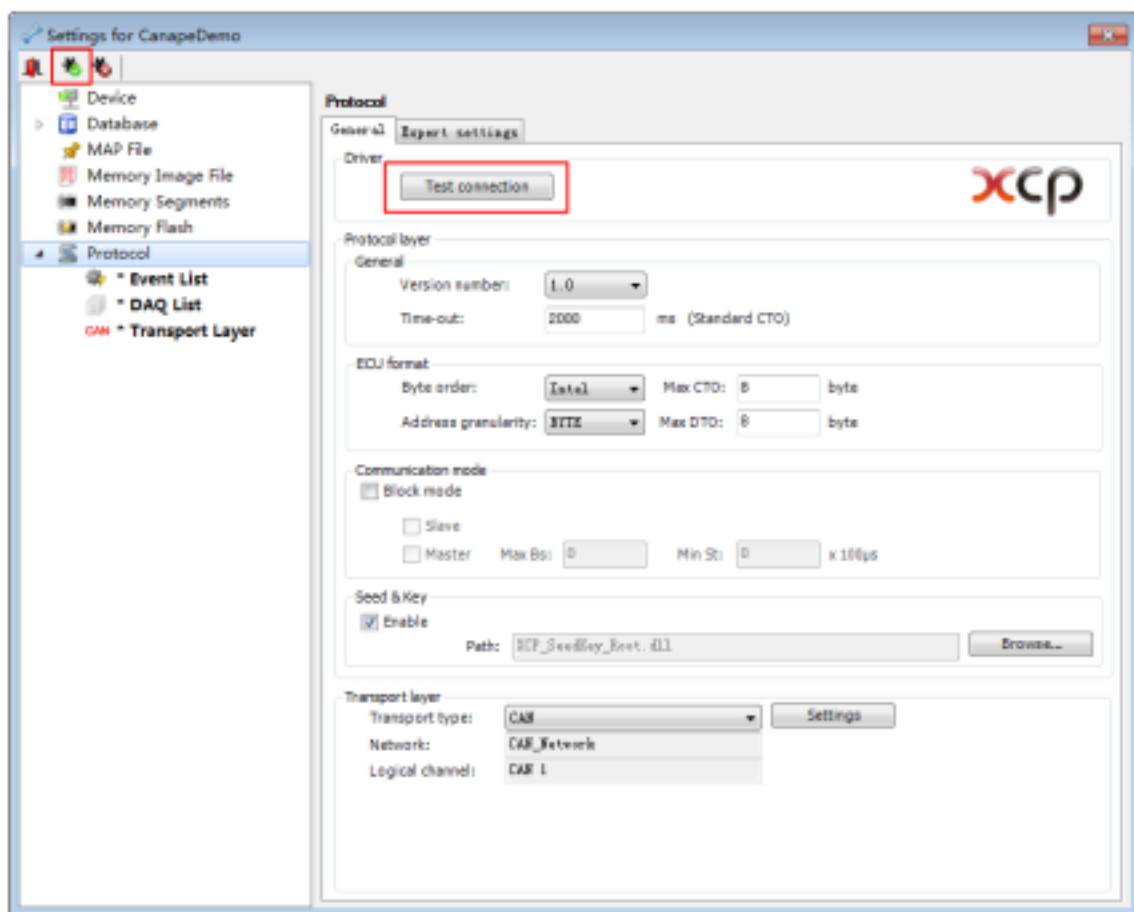
4, 打开 Change 设置为 500KB 的波特率 (根据项目实际使用的波特率来进行配置), 单击 OK, 完成设置。

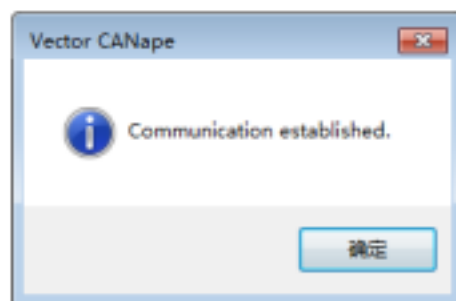


5, 点击 OK 之后返回到主界面, 点击 Devices->Devices Configuration, 如下图检查 Network 中设置是否和工程实际使用一直, 如果不一致, 请调整通道和波特率。



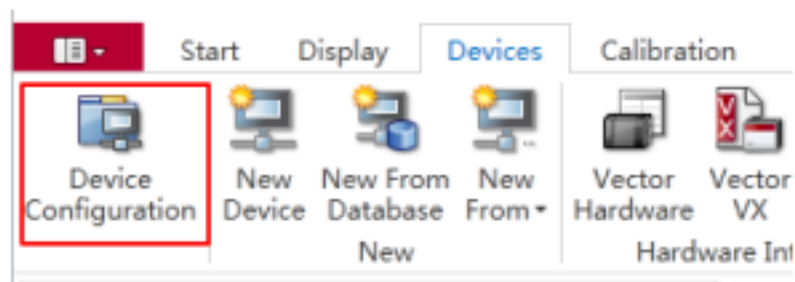
6, 回到 Protocol 下, 点击 Test connection, 出现下图界面说明 CANape 已经和板卡通讯成功, 接下来可以进行测量与标定。单击左上角的 Accept all changes 保存设置。保存完成后, 自动回到主界面。



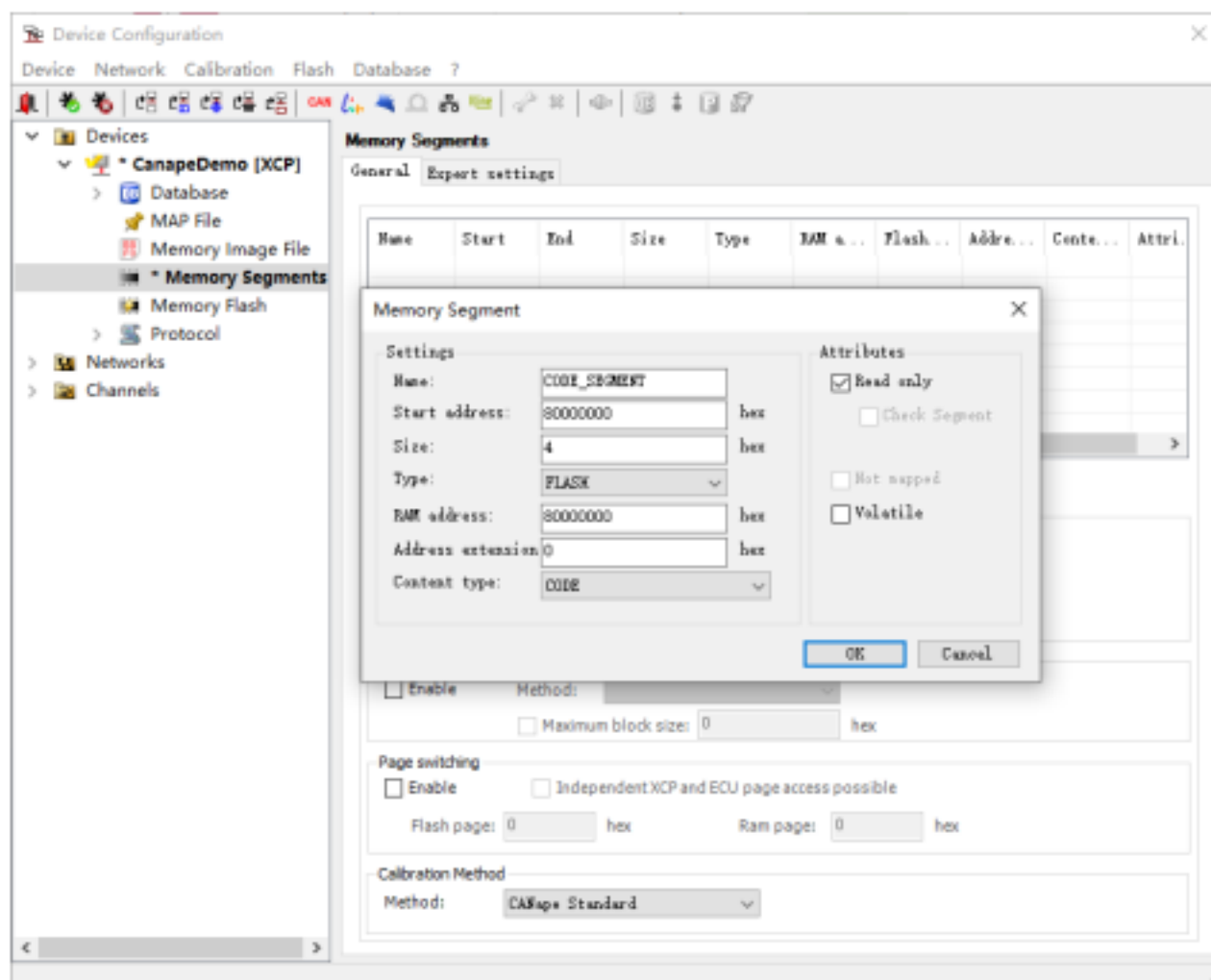


6.5 内存分布配置

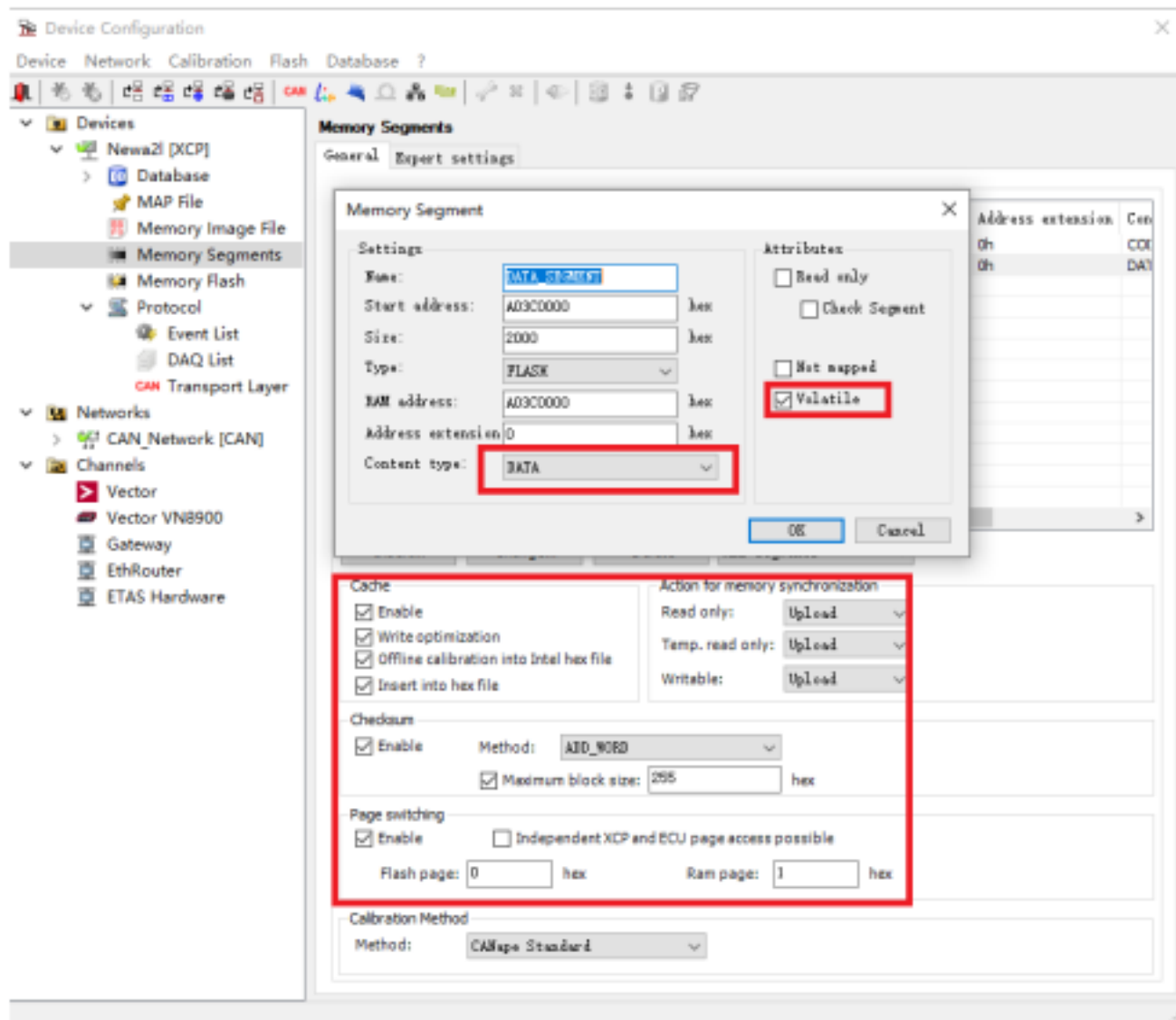
1, 点击 Device->Device Configuration.



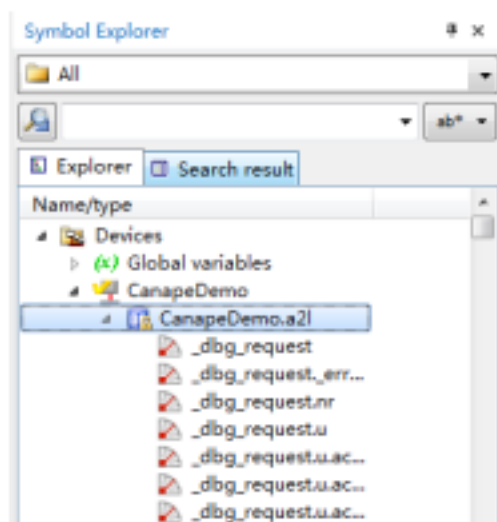
2, 选择 Devices, 选中当前 A2L 文件, 选择 MemorySegment, 设置 CODE_SEGMENT 地址和大小, 勾选 Read Only, Content Type 选择 CODE。此项根据工程 HEX 文件起始地址来进行更改。



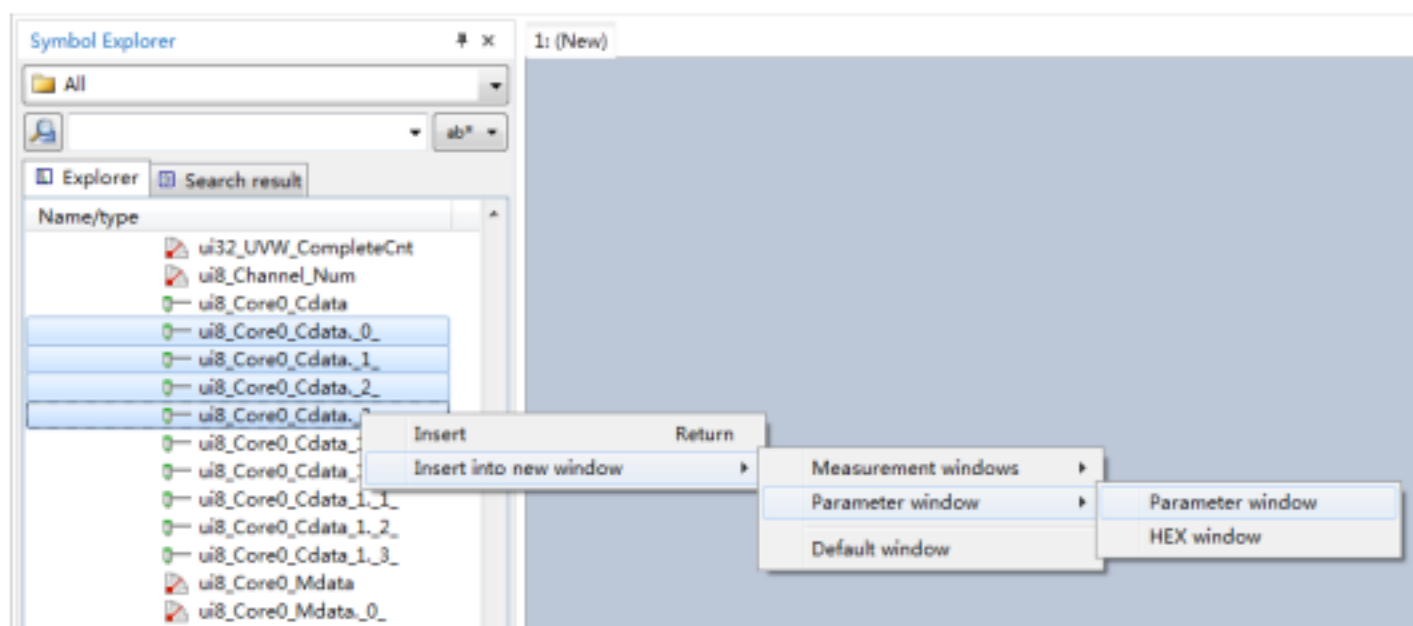
3, 其余标定区间配置, 按照工程实际情况配置。



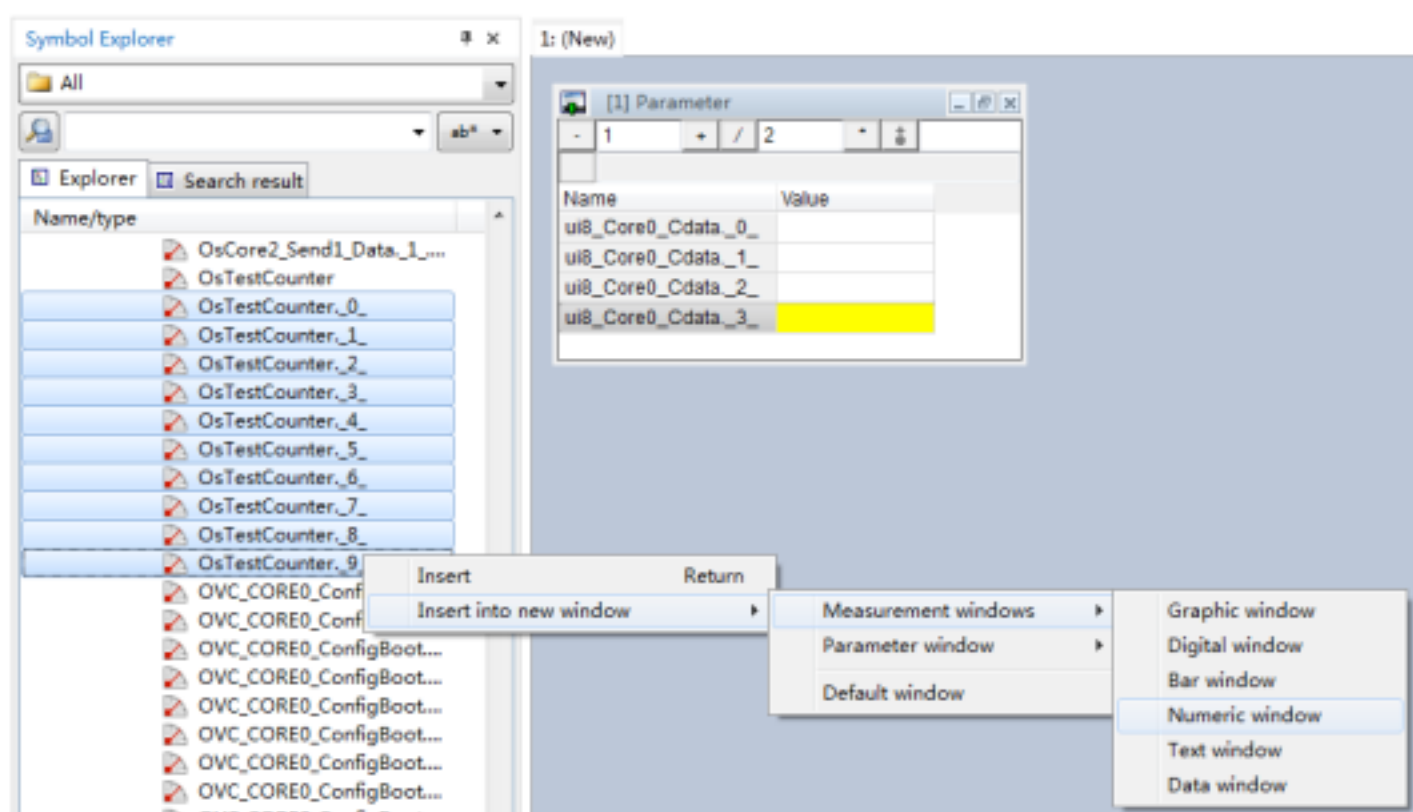
4, 在主界面的 Symbol explorer 窗口点击 Devices, 选择刚才新建的工程, 选择刚才加载的 A2L 文件, 选择检测量和标定量, 使用鼠标右键, 来进行添加窗口。



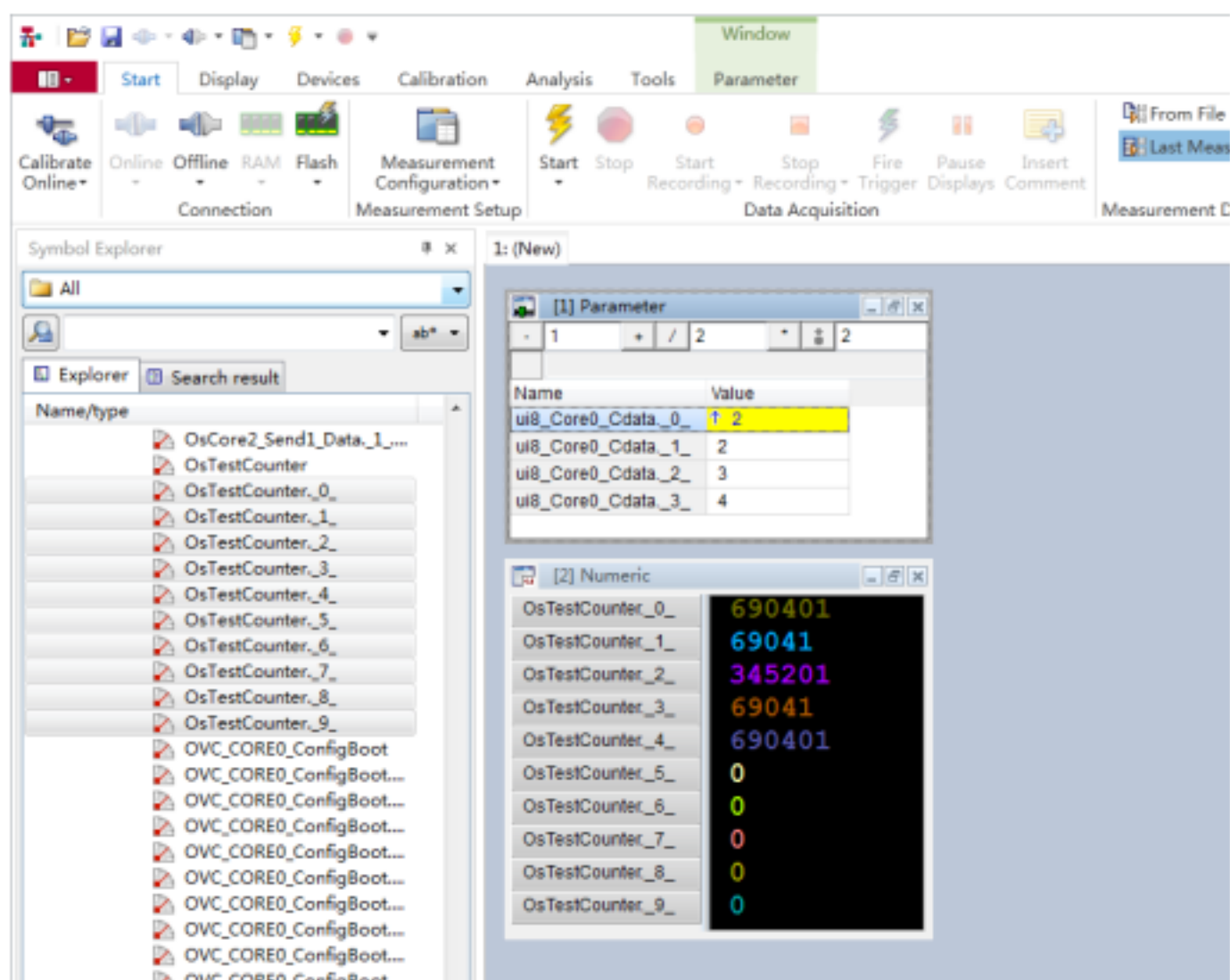
添加标定量



添加監測量



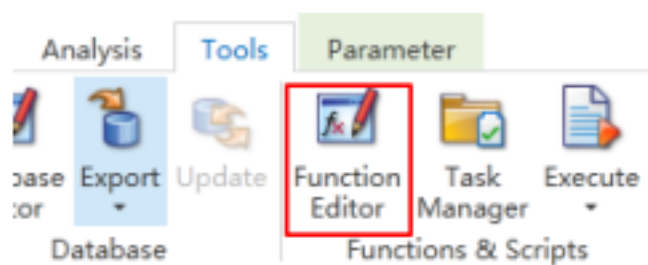
23, 按 Shift+F9, 即可开始进行标定和检测。



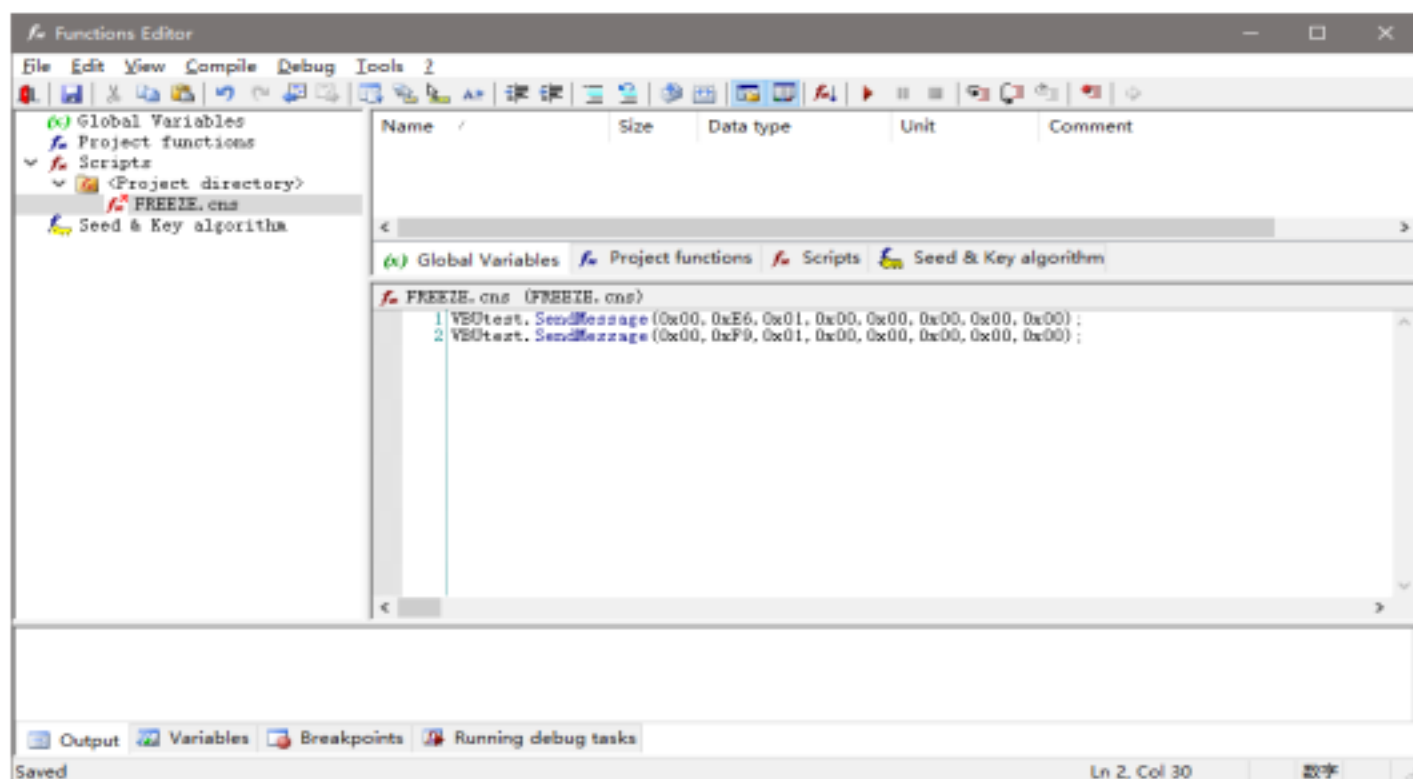
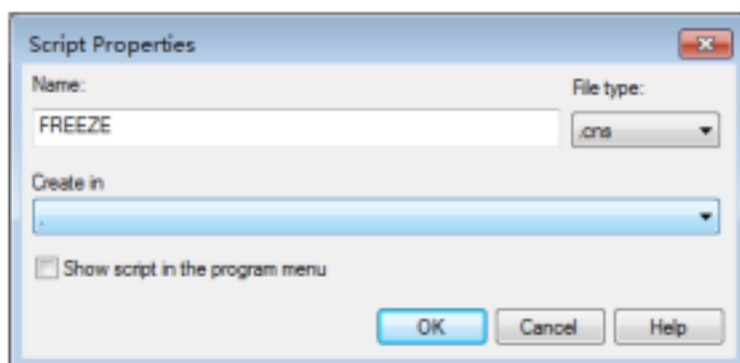
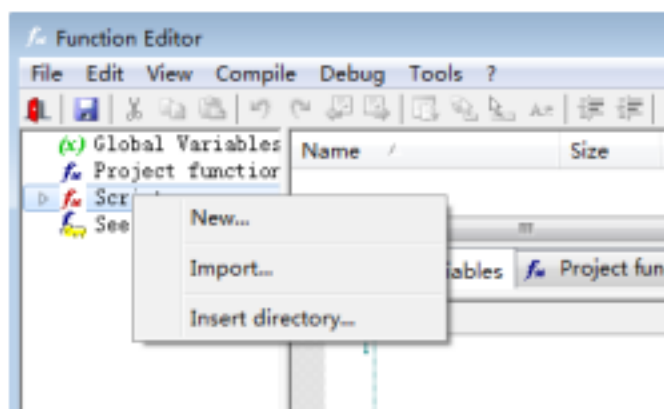
6.6 数据冻结（部分项目支持）

1, 冻结命令的制作。

点击 Tools, 选择 Functions editor

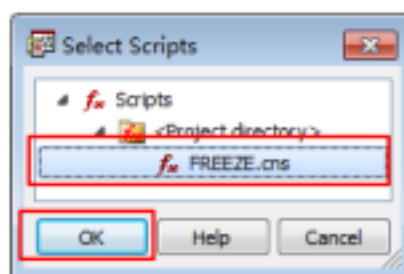
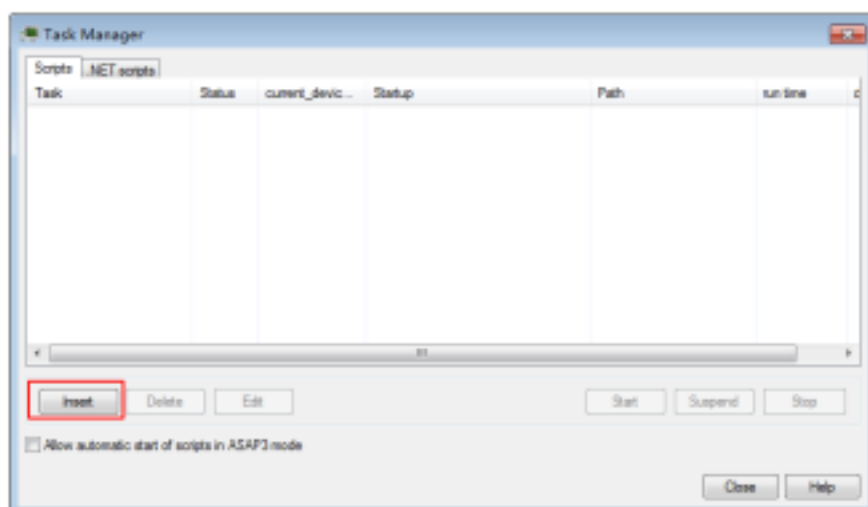
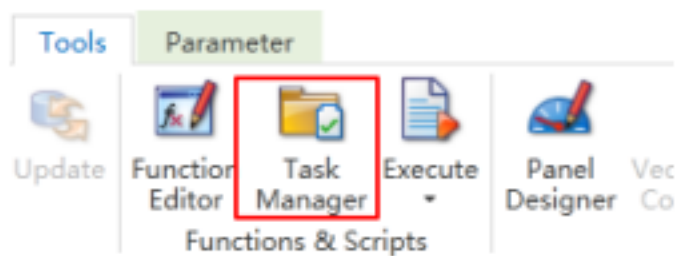


选择 Scripts 右键选择 NEW, 取名为 FREEZE, 输入下列命令, 点击保存

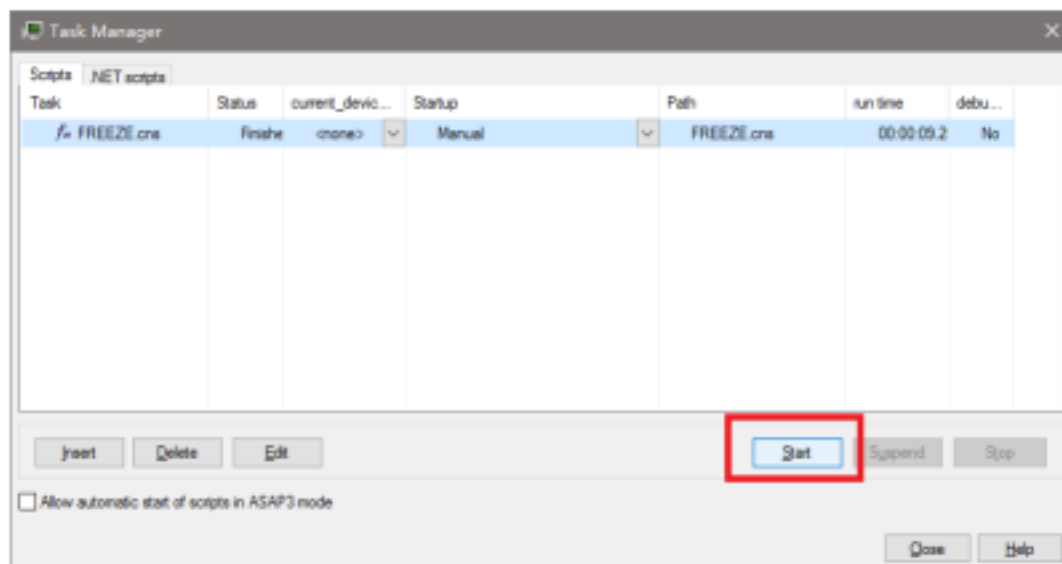


脚本中的“VBUtest”是 CANape 工程中 A2L 文件的名称。根据 A2L 名称来进行更改。

点击 Tools，选择 Task Manager

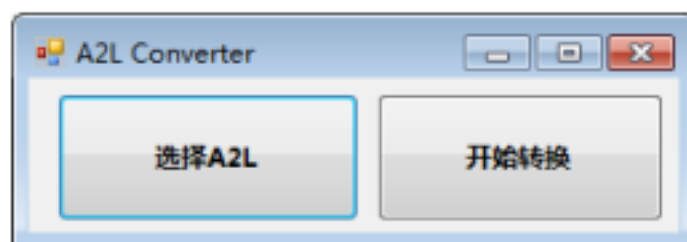


2, 发送冻结帧, 点击 Start 即可发送命令。



7 INCA 的 A2L 制作说明

在确认内存分布配置完成，CHECKSUM 模式配置完成后，即可使用 A2LConverter 工具转换，如下图打开 A2LConverter，点击“选择 A2L”，选择 A2L 文件，点击“打开”。



然后点击“开始转换”，会生成一个 INCA.A2L 的文件在当前目录下。

对比两个 A2L 文件，可以看到 Page 的定义不一样。（左：inca.a2l；右：Canape.a2l）

<pre> (block "PAGE" struct { uchar: /* PAGE_NUMBER: logical page number */ enum { "ECU_ACCESS_NOT_ALLOWED" = 0, "ECU_ACCESS_WITHOUT_ECU_ONLY" = 1, "ECU_ACCESS_WITH_ECU_ONLY" = 2, "ECU_ACCESS_DONT_CARE" = 3 }; /*The ECU_ACCESS_x flags indicate whether and how the ECU can access this page.*/ enum { "NOP_READ_ACCESS_NOT_ALLOWED" = 0, "NOP_READ_ACCESS_WITHOUT_ECU_ONLY" = 1, "NOP_READ_ACCESS_WITH_ECU_ONLY" = 2, "NOP_READ_ACCESS_DONT_CARE" = 3 }; /*The NOP_READ_ACCESS_x flags indicate whether and how the XCP master can access this page.*/ enum { "NOP_WRITE_ACCESS_NOT_ALLOWED" = 0, "NOP_WRITE_ACCESS_WITHOUT_ECU_ONLY" = 1, "NOP_WRITE_ACCESS_WITH_ECU_ONLY" = 2, "NOP_WRITE_ACCESS_DONT_CARE" = 3 }; /*The NOP_WRITE_ACCESS_x flags indicate whether the XCP master can write to this page.*/ taggedStruct { "TEXT_SEGMENT" uchar; /* references segment that initializes this page */ }; }; /* PAGEs for this SEGMENT */ ----- ECU_ACCESS_WITH_ECU_ONLY NOP_READ_ACCESS_WITH_ECU_ONLY NOP_WRITE_ACCESS_NOT_ALLOWED /end PAGE /begin PAGE 0x01 ECU_ACCESS_WITH_ECU_ONLY NOP_READ_ACCESS_WITH_ECU_ONLY NOP_WRITE_ACCESS_NOT_ALLOWED /end PAGE /end SEGMENT /end IF_DATA /end MEMORY_SEGMENT /begin MEMORY_SEGMENT VOLATILE_CAL_DATA "" DATA FLASH INTERM 0x00000000 0x2000 -1 -1 -1 -1 /begin IF_DATA XCP /begin SEGMENT 0x01 0x02 0x00 0x00 0x00 0x00 /begin CHECKSUM XCP_ADD_14 PAGE_BLOCK_SIZE 0x0256 EXTERNAL_FUNCTION "chksum.dll" /end CHECKSUM /begin PAGE 0x00 ECU_ACCESS_WITH_ECU_ONLY NOP_READ_ACCESS_WITH_ECU_ONLY NOP_WRITE_ACCESS_NOT_ALLOWED /end PAGE /begin PAGE 0x01 ECU_ACCESS_WITH_ECU_ONLY NOP_READ_ACCESS_WITH_ECU_ONLY NOP_WRITE_ACCESS_WITH_ECU_ONLY /end PAGE </pre>	<pre> (block "PAGE" struct { uchar: /* PAGE_NUMBER: logical page number */ enum { "ECU_ACCESS_NOT_ALLOWED" = 0, "ECU_ACCESS_ALLOWED_WITHOUT_ECU_ONLY" = 1, "ECU_ACCESS_ALLOWED_WITH_ECU_ONLY" = 2, "ECU_ACCESS_ALLOWED_DONT_CARE" = 3 }; /*The ECU_ACCESS_x flags indicate whether and how the ECU can access this page.*/ enum { "NOP_READ_ACCESS_NOT_ALLOWED" = 0, "NOP_READ_ACCESS_ALLOWED_WITHOUT_ECU_ONLY" = 1, "NOP_READ_ACCESS_ALLOWED_WITH_ECU_ONLY" = 2, "NOP_READ_ACCESS_ALLOWED_DONT_CARE" = 3 }; /*The NOP_READ_ACCESS_x flags indicate whether and how the XCP master can access this page.*/ enum { "NOP_WRITE_ACCESS_NOT_ALLOWED" = 0, "NOP_WRITE_ACCESS_ALLOWED_WITHOUT_ECU_ONLY" = 1, "NOP_WRITE_ACCESS_ALLOWED_WITH_ECU_ONLY" = 2, "NOP_WRITE_ACCESS_ALLOWED_DONT_CARE" = 3 }; /*The NOP_WRITE_ACCESS_x flags indicate whether the XCP master can write to this page.*/ taggedStruct { "TEXT_SEGMENT" uchar; /* references segment that initializes this page */ }; }; /* PAGEs for this SEGMENT */ ----- ECU_ACCESS_ALLOWED_WITH_ECU_ONLY NOP_READ_ACCESS_ALLOWED_WITH_ECU_ONLY NOP_WRITE_ACCESS_NOT_ALLOWED /end PAGE /begin PAGE 0x01 ECU_ACCESS_ALLOWED_WITH_ECU_ONLY NOP_READ_ACCESS_ALLOWED_WITH_ECU_ONLY NOP_WRITE_ACCESS_NOT_ALLOWED /end PAGE /end SEGMENT /end IF_DATA /end MEMORY_SEGMENT /begin MEMORY_SEGMENT VOLATILE_CAL_DATA "" DATA FLASH INTERM 0x00000000 0x2000 -1 -1 -1 -1 /begin IF_DATA XCP /begin SEGMENT 0x01 0x02 0x00 0x00 0x00 0x00 /begin CHECKSUM XCP_ADD_14 PAGE_BLOCK_SIZE 0x0256 EXTERNAL_FUNCTION "chksum.dll" /end CHECKSUM /begin PAGE 0x00 ECU_ACCESS_ALLOWED_DONT_CARE NOP_READ_ACCESS_ALLOWED_DONT_CARE NOP_WRITE_ACCESS_NOT_ALLOWED /end PAGE /begin PAGE 0x01 ECU_ACCESS_ALLOWED_DONT_CARE NOP_READ_ACCESS_ALLOWED_DONT_CARE NOP_WRITE_ACCESS_ALLOWED_DONT_CARE /end PAGE </pre>
--	--

8 文件修订履历表

版次	修订内容	修订日期	修订人	备注
V1.00	初始版本	2016/11/24	苗清	
V1.01	增加 A2L 制作细节 增加 INCA 的 A2L 制作说明	2019/04/02	邢瑞东	
V1.02	增加第 4 章节	2019/10/06	邢瑞东	
V1.03	更换文档模板，重新规划章节，更新部分 图片为 CANape16SP16，删除 INCA 的 A2L 制作说明部分。	2019/11/06	苗清	
V1.04	添加第七章：INCA 的 A2L 制作说明	2020/07/09	洪超	
V1.05				
V1.06	更换文档模板，重新排版	2021/01/13	苗清	